第六章
基于异构系统的并行计算

哈尔滨工业大学
张伟哲

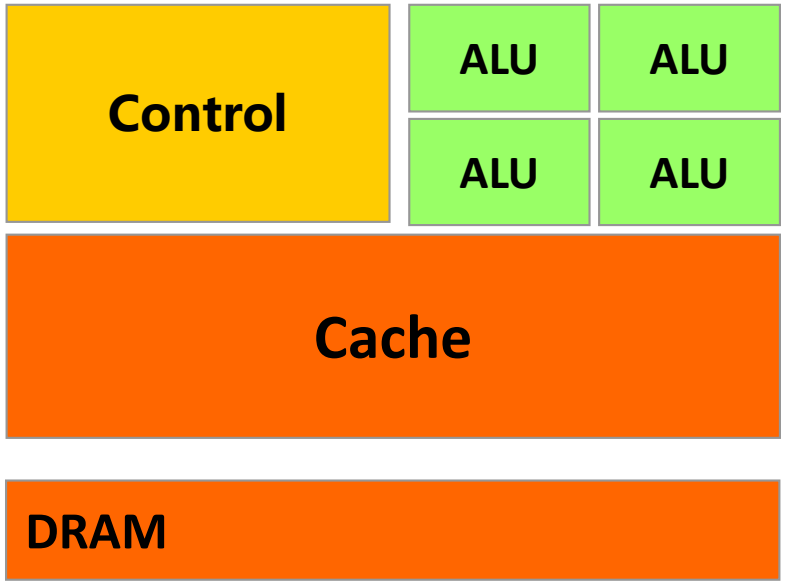
2024, Fall Semester

什么是GPU计算

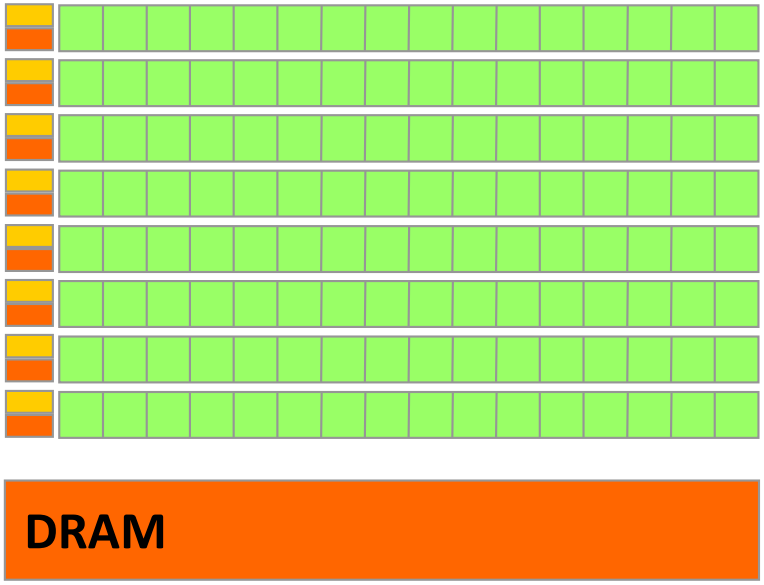
- GPU（Graphic Processing Unit）：从图形处理到通用计算
- GPU为高度并行的实时3D渲染计算而设计，高GFLOPS，高带宽
- 3D渲染技术及3D API的发展，促进GPU向通用计算处理器发展
- NVIDIA GPU为通用计算专门优化设计，于2007年推出CUDA（Compute Unified Device Architecture）

GPU与CPU硬件架构的对比

- 与CPU相比，GPU具有更多的计算单元，但对于复杂控制过程的处理能力则比CPU差
- GPU更适用于同时进行大量简单的统一操作



CPU



GPU

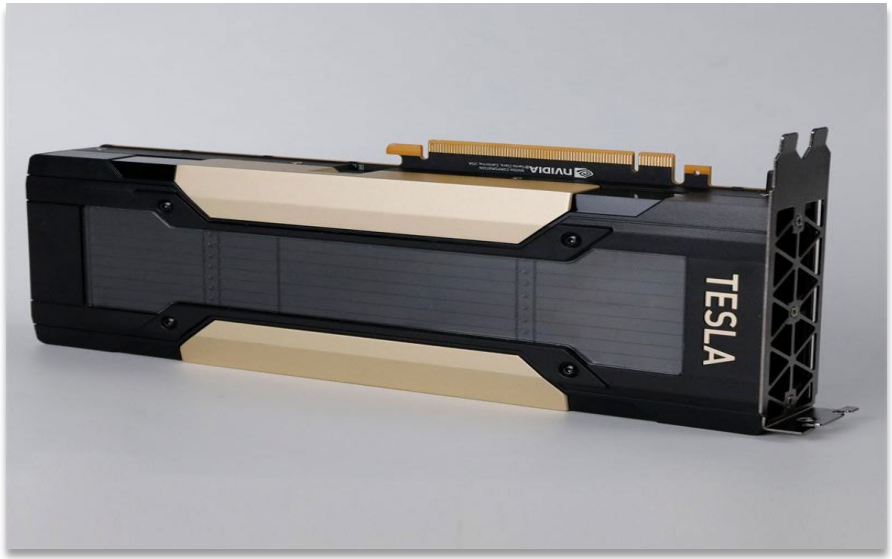
GPU与CPU硬件架构的对比

CPU (Skylake)

- 40 cores (2 20-core chips)
- 2 threads each
- 2xAVX-512, so 2x8 in double precision
- ~640 way parallelism (40*2*8)

GPU (V100)

- 80 SM
- 64 warps per SM
- 32 threads per warp in double precision
- ~150,000+ way parallelism (80*64*32)

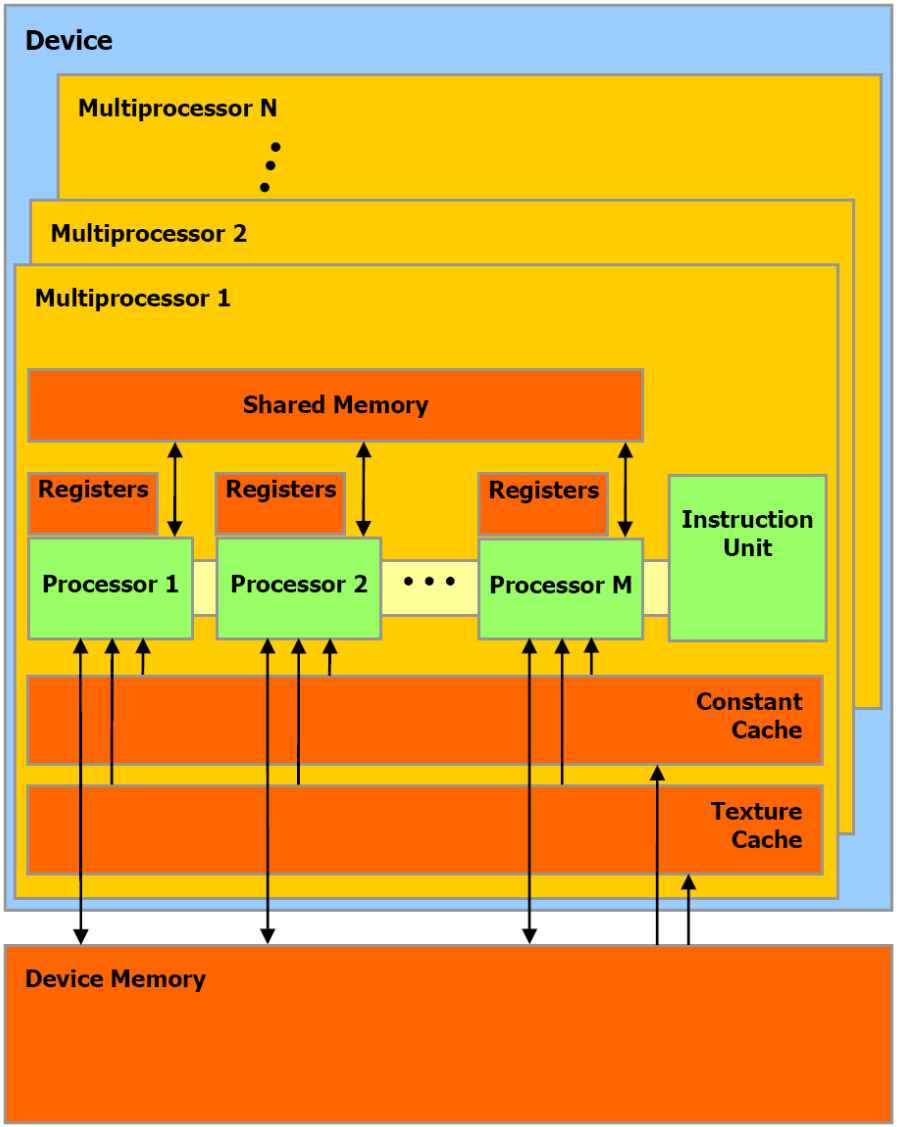


GPU与CPU应用范围对比

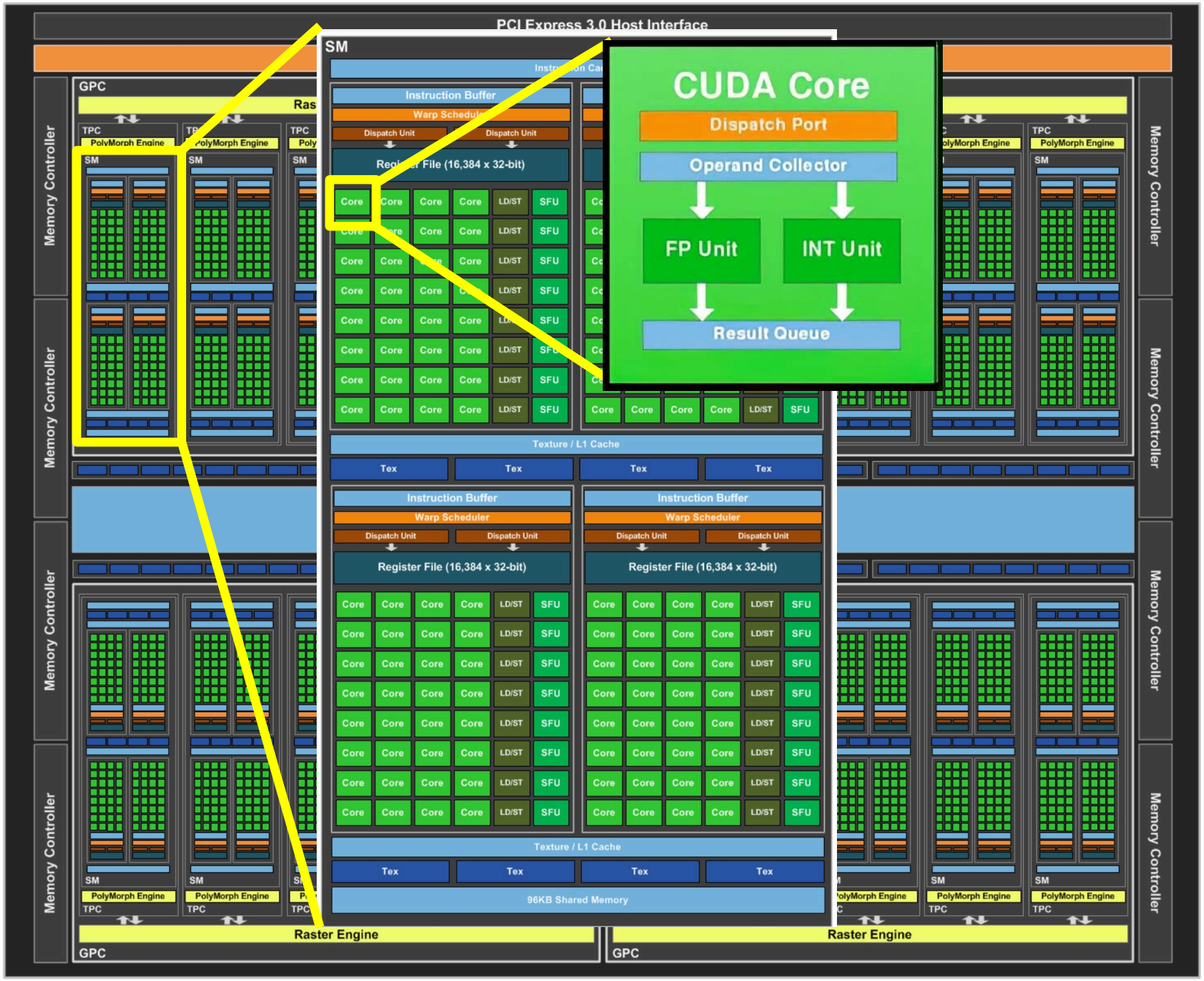
- CPU: control processor
  - 不规则数据结构
  - 不可预测存取模式
  - 递归算法
  - 分支密集型算法
  - 单线程程序
- GPU: data processor
  - 规则数据结构
  - 可预测存取模式
  - 油气勘探、金融分析、医疗成像、有限元、基因分析、地理信息系统、深度学习…

GPU基本硬件架构

- 流多处理器（SM，Stream Multiprocessor）
- 流处理器（SP，Stream Processor）
- 共享内存（Shared Memory）
- 板载显存（Device Memory）

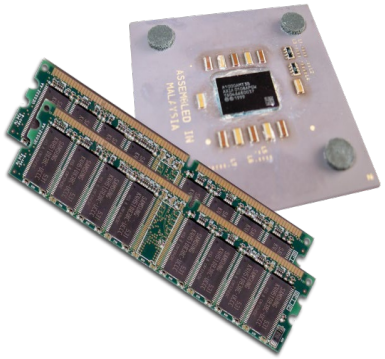


PCI Express 3.0 Host Interface
GigaThread Engine
GPC
Raster Engine
TPC
PolyMorph Engine
SM
Memory Controller
L2 Cache


PCI Express 3.0 Host Interface
GPC
TPC
PolyMorph Engine
SM
Memory Controller
Raster Engine
SM
Instruction Buffer
Warp Scheduler
Dispatch Unit
Register File (16,384 x 32-bit)
Core
LD/ST
SFU
Texture / L1 Cache
Tex
96KB Shared Memory
CUDA Core
Dispatch Port
Operand Collector
FP Unit
INT Unit
Result Queue

异构计算

- GPU并不是独立运行的计算平台，需要与CPU协同工作，可以看成CPU的协处理器
- GPU计算，实际是**基于CPU+GPU的异构计算**
- 在异构计算架构中，GPU与CPU通过PCIe总线连接，CPU所在位置是主机端（host），GPU所在位置称为设备端（device）



host

PCIe Bus

device

异构计算

- GPU包括更多运算核心，适合数据并行的计算密集型任务，如大型矩阵运算
- CPU的运算核心较少，但是其可以实现复杂的逻辑运算，适合控制密集型任务
- CPU上的线程是重量级的，上下文切换开销大
- GPU由于存在众多核心，线程是轻量级的
- CPU+GPU的异构系统可以优势互补，CPU负责处理逻辑复杂的串行任务，GPU重点处理数据密集型的并行计算任务

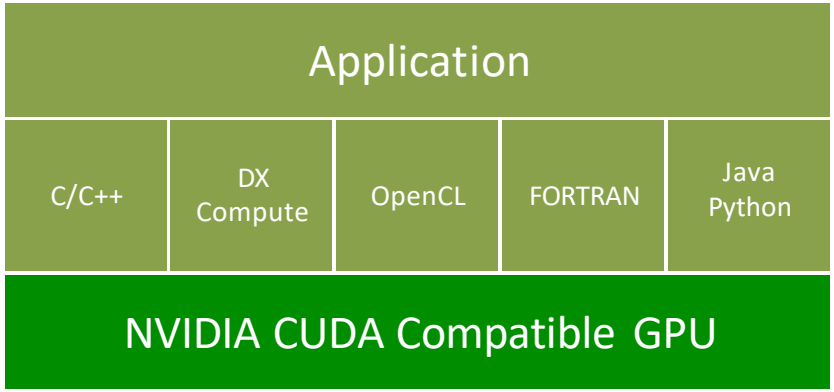
CUDA

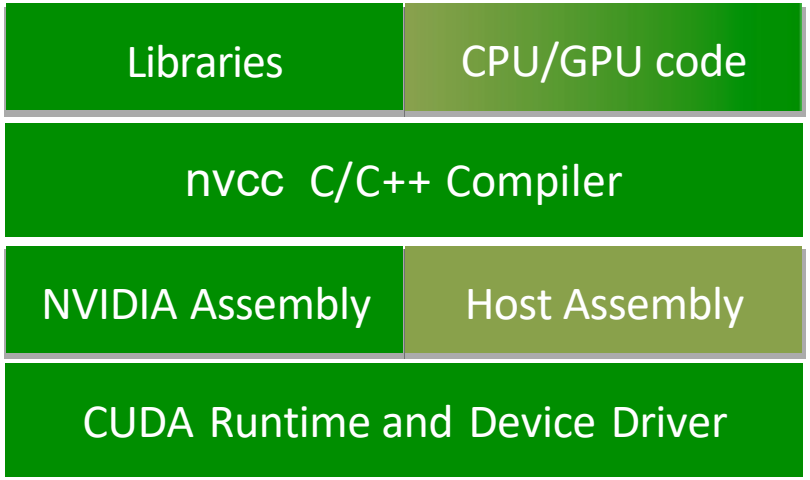
- CUDA是NVIDIA公司所开发的GPU编程模型
- 提供了GPU编程的简易接口，基于CUDA编程可以构建基于GPU计算的应用程序
- CUDA提供了对其它编程语言的支持，如C/C++，Python，Fortran等语言
- 硬件：NVIDIA GPU，可以是专门的科学计算卡，比如Tesla A100），也可以是普通的游戏显卡，比如GTX或者RTX
- 软件：CUDA Toolkit：Windows、Mac 和大多数标准 Linux 发行版都支持 https://developer.nvidia.com/cuda-toolkit

CUDA

CUDA Device Driver
CUDA Toolkit (compiler, debugger, profiler, lib)
CUDA SDK (examples)
Windows, Mac OS, Linux

Parallel Computing Architecture





Libraries – FFT, Sparse Matrix, BLAS, RNG, CUSP, Thrust…

异构计算

```
#include <iostream>
#include <algorithm>

using namespace std;

#define N        1024
#define RADIUS     3
#define BLOCK_SIZE 16

__global__ void stencil_1d(int *in, int *out) {
        __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];
        int gindex = threadIdx.x + blockIdx.x * blockDim.x;
        int lindex = threadIdx.x + RADIUS;

        // Read input elements into shared memory
        temp[lindex] = in[gindex];
        if (threadIdx.x < RADIUS) {
                temp[lindex - RADIUS] = in[gindex - RADIUS];
                temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
        }

        // Synchronize (ensure all the data is available)
        __syncthreads();

        // Apply the stencil
        int result = 0;
        for (int offset = -RADIUS ; offset <= RADIUS ; offset++)
                result += temp[lindex + offset];

        // Store the result
        out[gindex] = result;
}

void fill_ints(int *x, int n) {
        fill_n(x, n, 1);
}

int main(void) {
        int *in, *out;          // host copies of a, b, c
        int *d_in, *d_out;      // device copies of a, b, c
        int size = (N + 2*RADIUS) * sizeof(int);

        // Alloc space for host copies and setup values
        in  = (int *)malloc(size); fill_ints(in,  N + 2*RADIUS);
        out = (int *)malloc(size); fill_ints(out, N + 2*RADIUS);

        // Alloc space for device copies
        cudaMalloc((void **)&d_in, size);
        cudaMalloc((void **)&d_out, size);

        // Copy to device
        cudaMemcpy(d_in,  in,  size, cudaMemcpyHostToDevice);
        cudaMemcpy(d_out, out, size, cudaMemcpyHostToDevice);

        // Launch stencil_1d() kernel on GPU
        stencil_1d<<<N/BLOCK_SIZE,BLOCK_SIZE>>>(d_in + RADIUS,
d_out + RADIUS);

        // Copy result back to host
        cudaMemcpy(out, d_out, size, cudaMemcpyDeviceToHost);

        // Cleanup
        free(in); free(out);
        cudaFree(d_in); cudaFree(d_out);
        return 0;
}
```

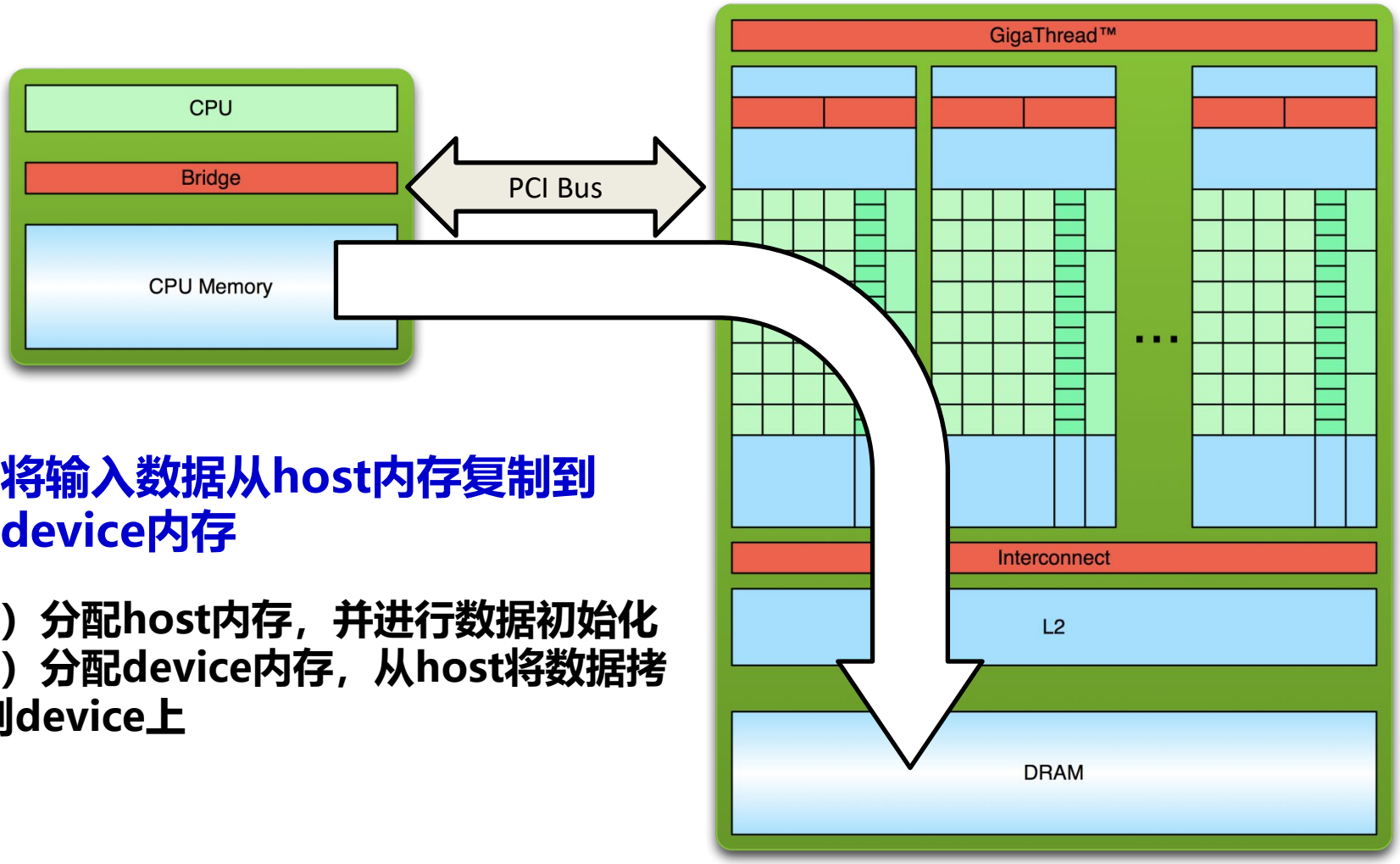
parallel fn

serial code

parallel code

serial code

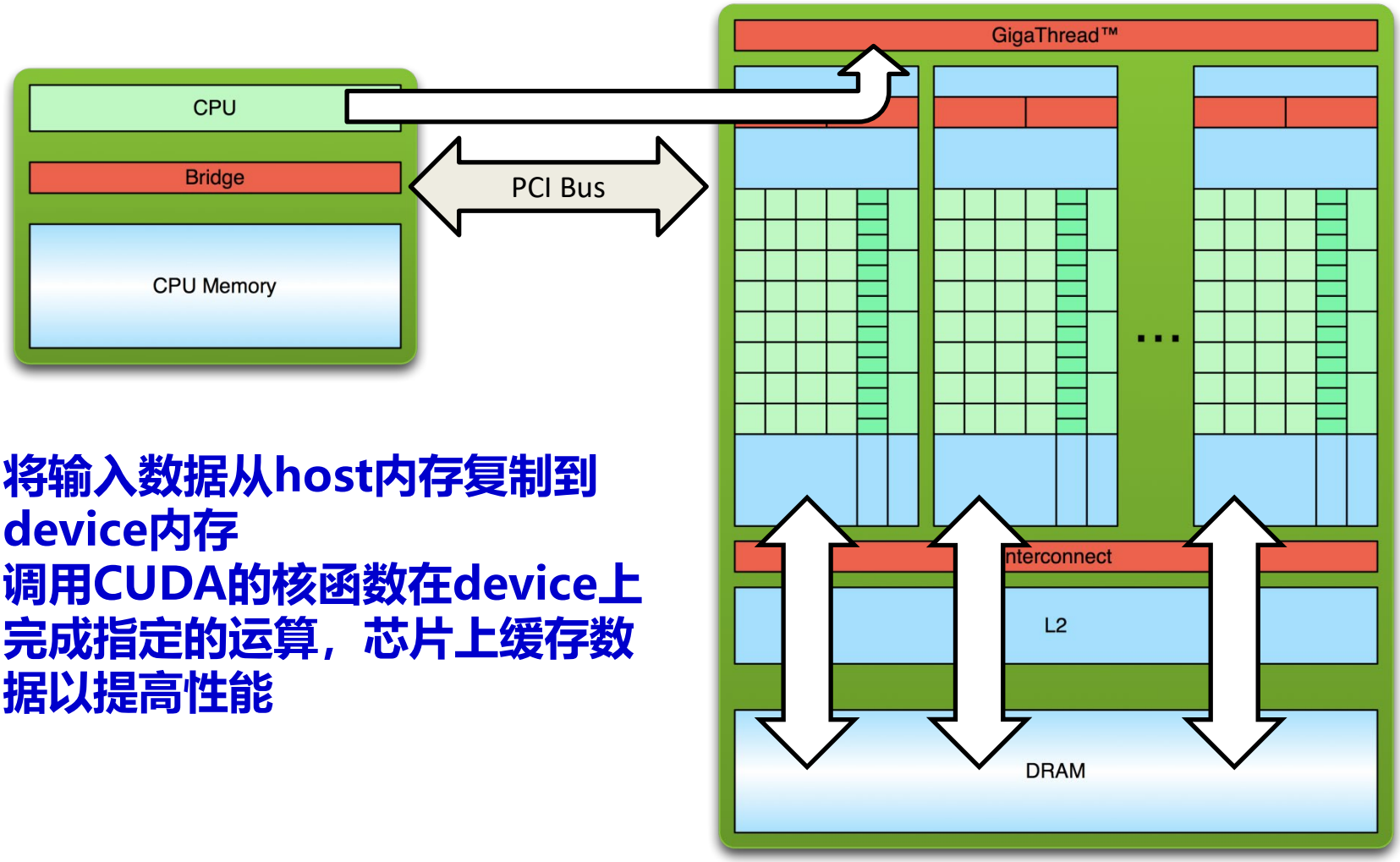
CPU-GPU协同处理流程



1. 将输入数据从host内存复制到device内存

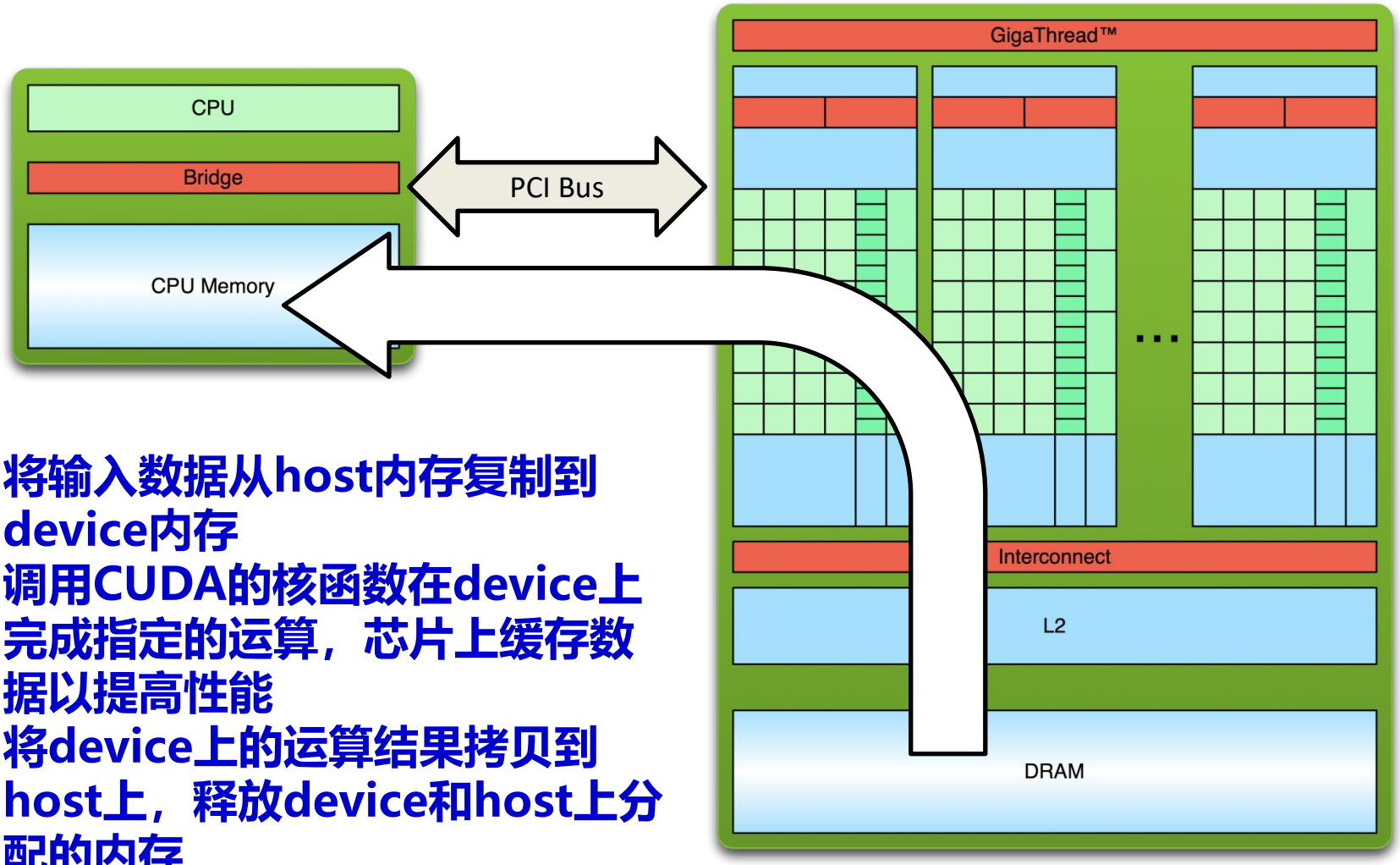
（1）分配host内存，并进行数据初始化
（2）分配device内存，从host将数据拷贝到device上

CPU-GPU协同处理流程



1. 将输入数据从host内存复制到device内存
2. 调用CUDA的核函数在device上完成指定的运算，芯片上缓存数据以提高性能

CPU-GPU协同处理流程



1. 将输入数据从host内存复制到device内存
2. 调用CUDA的核函数在device上完成指定的运算，芯片上缓存数据以提高性能
3. 将device上的运算结果拷贝到host上，释放device和host上分配的内存

Hello World!

```
int main(void) {
    printf("Hello World!\n");
    return 0;
}
```

- 标准C代码运行在host上
- NVIDIA编译器（nvcc）可用于编译没有device代码的程序

Output:

```
$ nvcc hello_world.cu
$./a.out
Hello World!
$
```

Hello World！ with Device Code

```
__global__ void mykernel(void) {
}

int main(void) {
    mykernel<<<1,1>>>();
    printf("Hello World!\n");
    return 0;
}
```

- 包含两个新的语法元素…

CUDA函数定义

```
__global__ void mykernel(void) {
}
```

- CUDA C/C++关键字`__global__`表示一个kernel函数:
  - 在device上运行
  - 从host代码调用
- nvcc将源代码分为host部分和device部分
  - device函数（例如`mykernel()`）由NVIDIA编译器编译
  - host函数（例如`main()`）标准主机编译器编译，如gcc

CUDA函数定义

	执行	调用
__device__ float DeviceFunc()	Device	Device
__global__ void KernelFunc()	Device	Host
__host__ float HostFunc()	Host	Host

- __global__定义kernel函数
  - 必须返回void
  - kernel 函数都是异步执行
- __device__和__host__ 可以组合使用
  - 则被定义的函数在CPU和GPU上都被编译
- 不添加限定词时，函数默认为__host__

Hello World！with Device Code

```
mykernel<<<1,1>>>();
```

- <<< >>>尖括号表示从host代码到device代码的调用
  - 也称为“kernel启动”，异步
  - 里面的参数(1,1)，分别表示线程块的个数、每个线程块中线程的个数，乘积就是启动的线程数

这就是在GPU上执行函数所需的全部内容！

Hello World！ with Device Code

```
__global__ void mykernel(void){
}

int main(void) {
      mykernel<<<1,1>>>();
      printf("Hello World!\n");
      return 0;
}
```

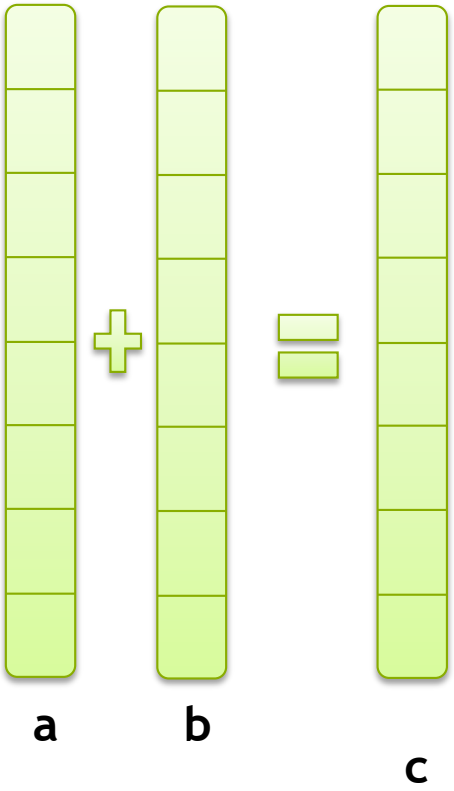
Output:

```
$ nvcc hello.cu
$ a.out
Hello World!
$
```

mykernel()什么都没做

Vector Addition

- GPU是适合大规模并行的
- 我们从两个整数相加开始，然后构建向量加法



Addition on the Device

- 将两个整数相加的简单kernel

```
__global__ void add(int *a, int *b, int *c) {
        *c = *a + *b;
}
```

- `__global__`是CUDA C/C++关键字
  - `add()`将在device端执行
  - `add()`将从host端调用

Addition on the Device

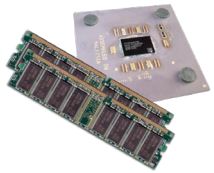
- kernel函数中变量是指针类型

```
__global__ void add(int *a, int *b, int *c) {
        *c = *a + *b;
}
```

- add()在device上运行，a、b和c必须指向device内存
- 因此，需要在GPU上分配内存

内存管理

- host内存和device内存是独立的实体
  - device指针指向GPU内存
  - host指针指向CPU内存



- 用于处理device内存的CUDA API
  - `cudaMalloc()`, `cudaFree()`, `cudaMemcpy()`
  - 类似C语言的`malloc()`, `free()`, `memcpy()`

```
cudaError_t cudaMalloc(void** devPtr, size_t size)
```

```
cudaError_t cudaFree ( void* devPtr )
```

内存管理

```
cudaError_t cudaMemcpy(void *dist, const void* src,
                       size_t count, cudaMemcpyKind kind)
```

- 在 host 和device之间复制数据，cudaMemcpyKind 指定复制的方向

kind	含义
cudaMemcpyDeviceToHost	从设备向主机拷贝
cudaMemcpyHostToDevice	从主机向设备拷贝
cudaMemcpyHostToHost	从主机向主机拷贝
cudaMemcpyDeviceToDevice	从设备向设备拷贝

```
cudaError_t cudaMemcpyAsync(void *dist, const void* src,
                            size_t count, cudaMemcpyKind kind,
                            cudaStream_t stream = 0)
```

- 异步数据传输，可以在调用函数后立即返回，无需等待数据传输完成

Addition on the Device

- add()核函数

```
__global__ void add(int *a, int *b, int *c) {
        *c = *a + *b;
}
```

- main()函数

```
int main(void) {
     int a, b, c;              // host copies of a, b, c
     int *d_a, *d_b, *d_c;   // device copies of a, b, c
     int size = sizeof(int);

     // Allocate space for device copies of a, b, c
     cudaMalloc((void **)&d_a, size);
     cudaMalloc((void **)&d_b, size);
     cudaMalloc((void **)&d_c, size);

     // Setup input values
     a = 2;
     b = 7;
```

Addition on the Device

- **main()函数**

```
        // Copy inputs to device
        cudaMemcpy(d_a, &a, size, cudaMemcpyHostToDevice);
        cudaMemcpy(d_b, &b, size, cudaMemcpyHostToDevice);

        // Launch add() kernel on GPU
        add<<<1,1>>>(d_a, d_b, d_c);

        // Copy result back to host
        cudaMemcpy(&c, d_c, size, cudaMemcpyDeviceToHost);

        printf("%d\n", c);
        // Cleanup
        cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
        return 0;
}
```

开始并行化

- 如何在device上并行运行代码

```
add<<< 1, 1 >>>();
```

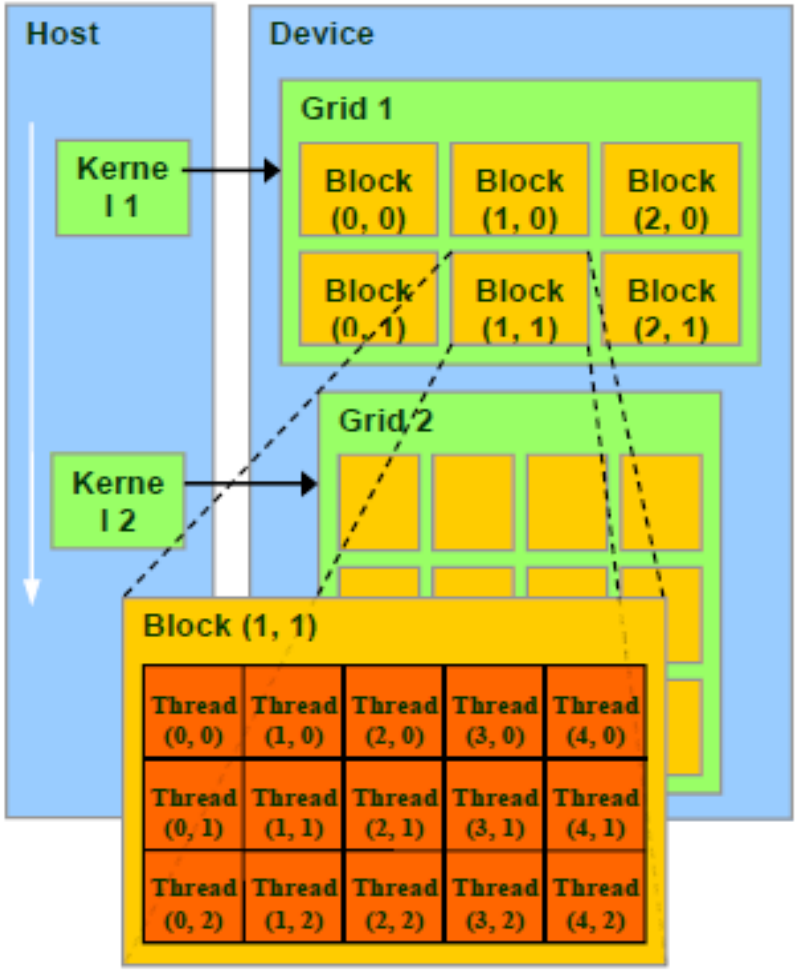
⇓

```
add<<< N, 1 >>>();
```

- 从执行add()函数1次，变为并行执行N次
- 通过并行运行add()，可以进行向量加法

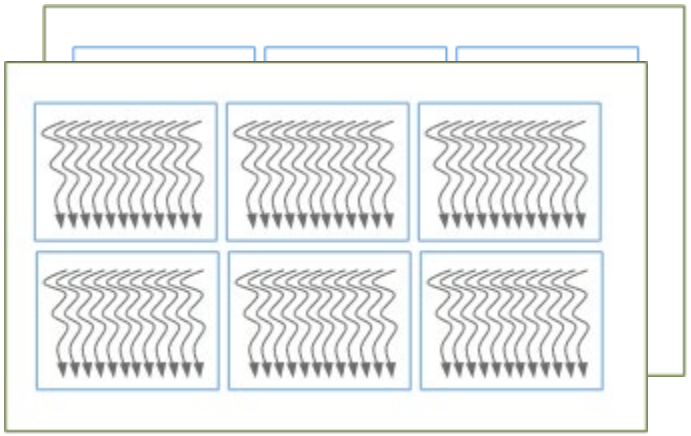
线程组织模型

- 每个Kernel 函数包含非常多的线程，对应一个Grid
- 所有Thread执行相同顺序程序
- Thread被划分成线程块 Block
- 同一Block内的线程可以通过共享SM资源或同步相互协作
- 每个Thread和Block拥有唯一ID，通过内置变量读取
- 层次结构：
  Grid → Block → Thread



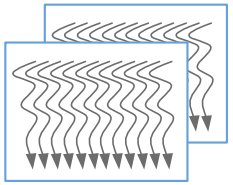
线程与硬件的关系

Grids



Threads

Blocks

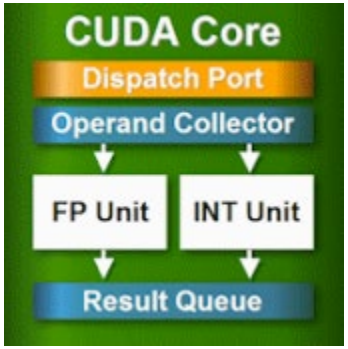


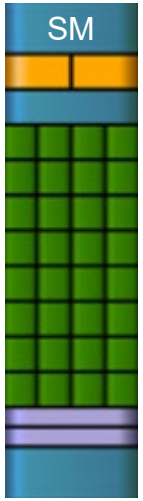
512-1024 threads / block

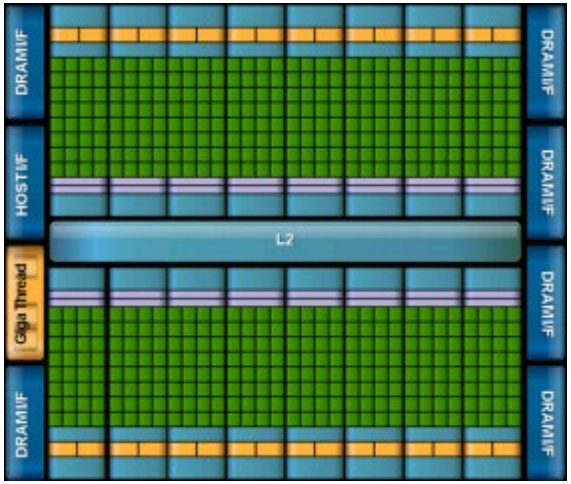
Kernel is executed by threads processed by CUDA Core

Maximum 8 blocks per SM
32 parallel threads are executed at the same time in a **Warp**

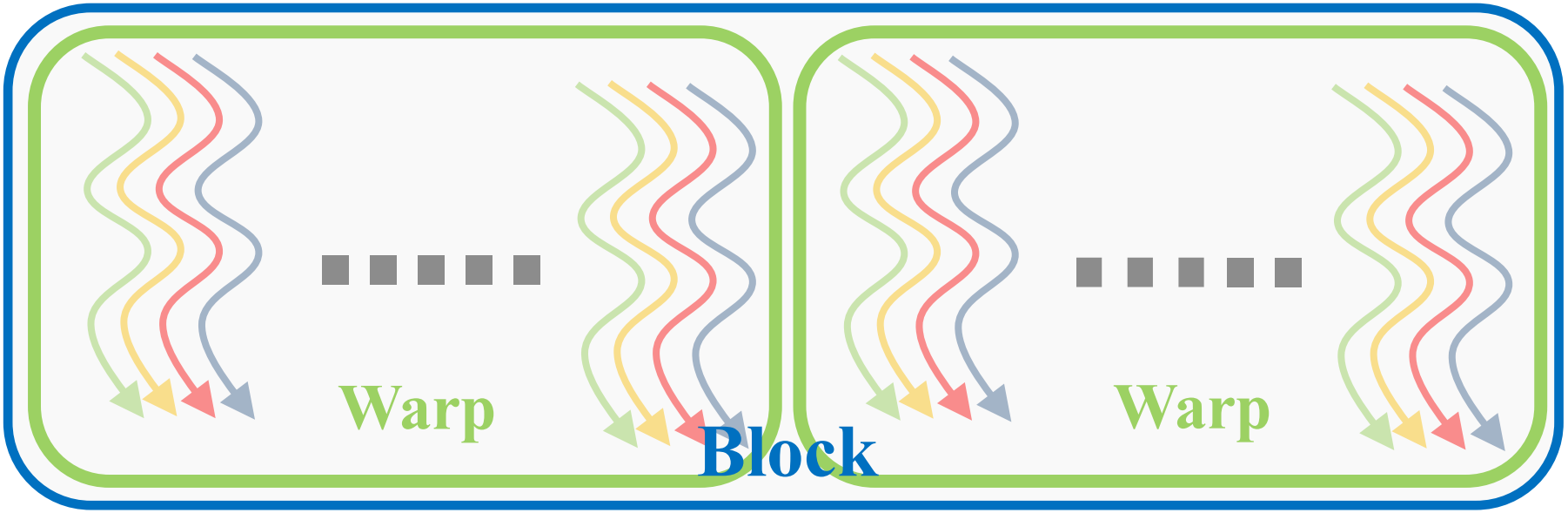
One grid per kernel with multiple concurrent kernels

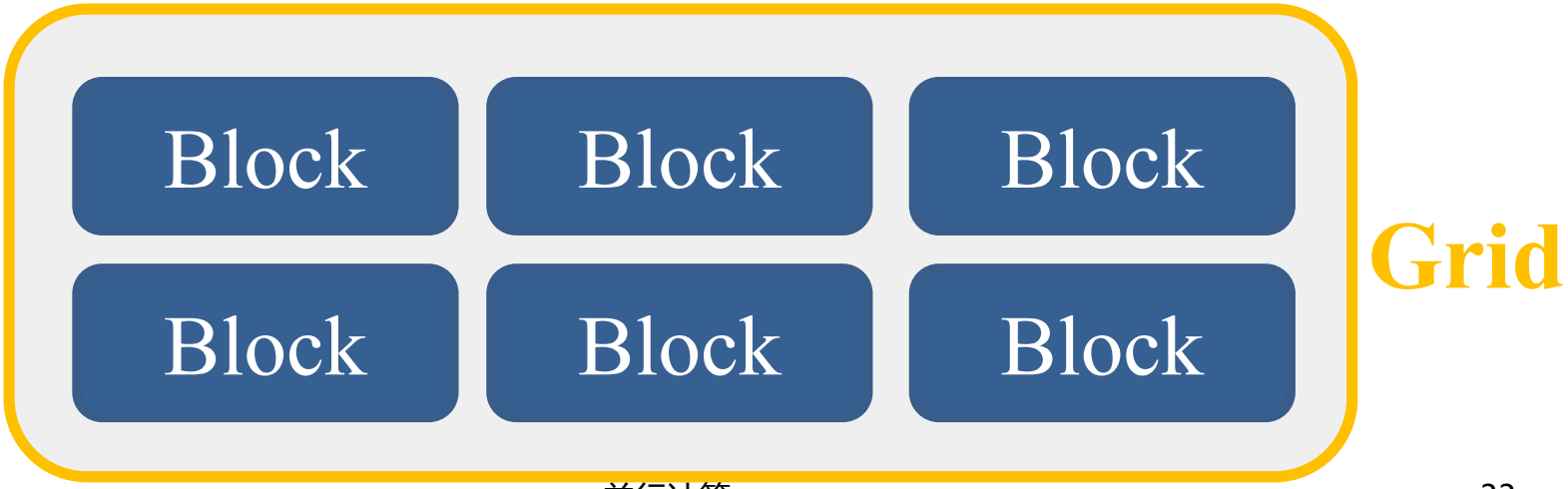




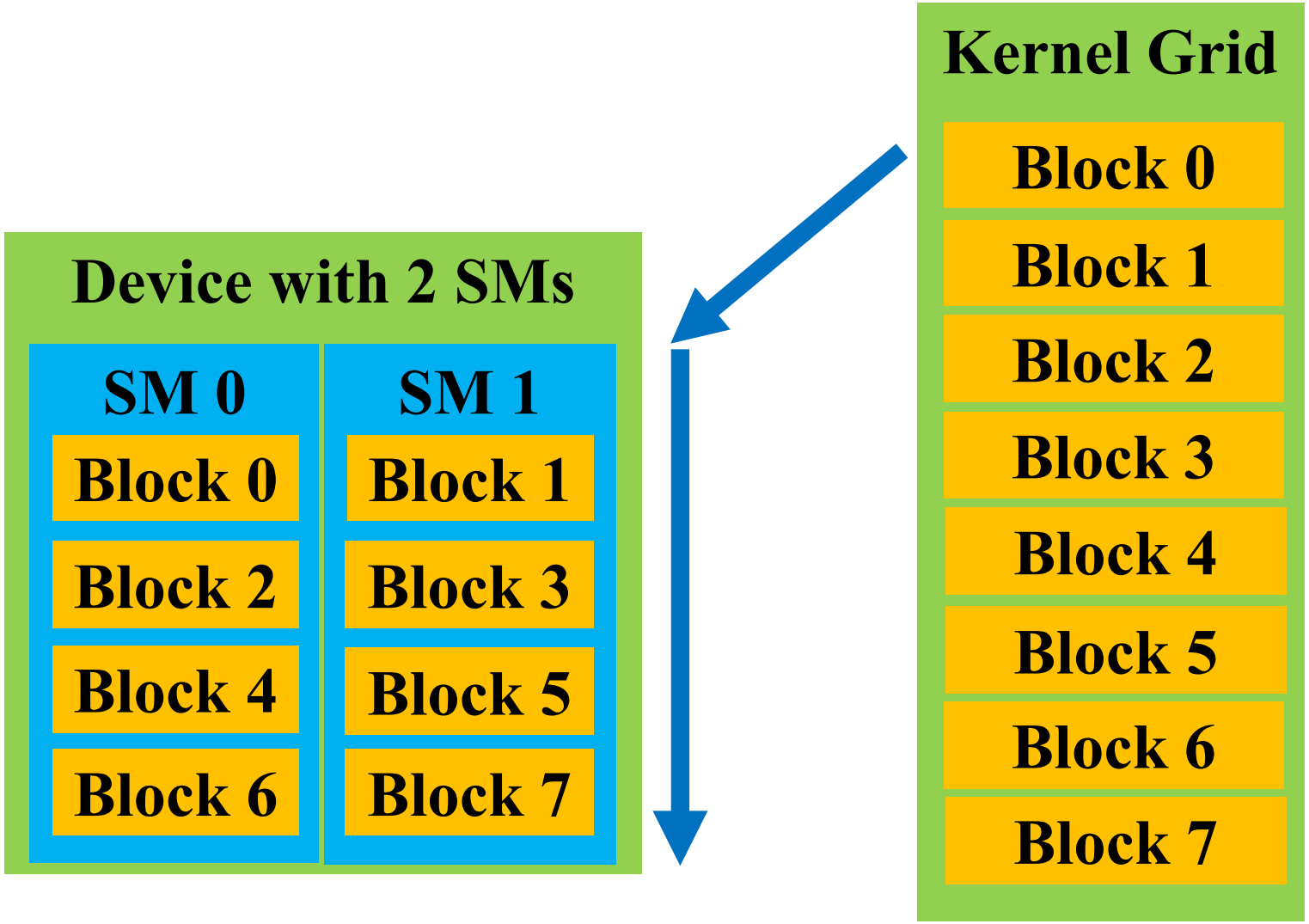


线程与硬件的关系

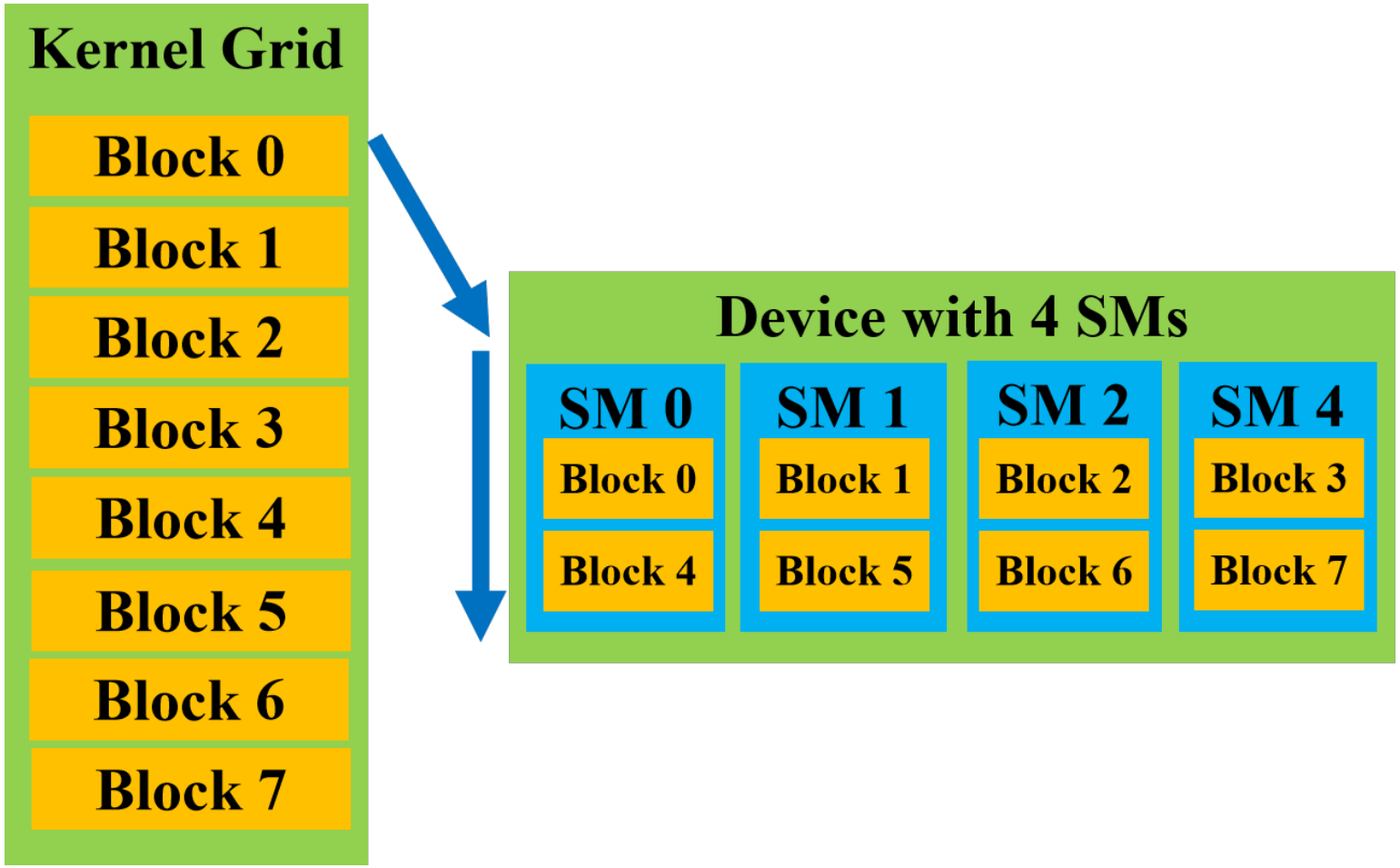




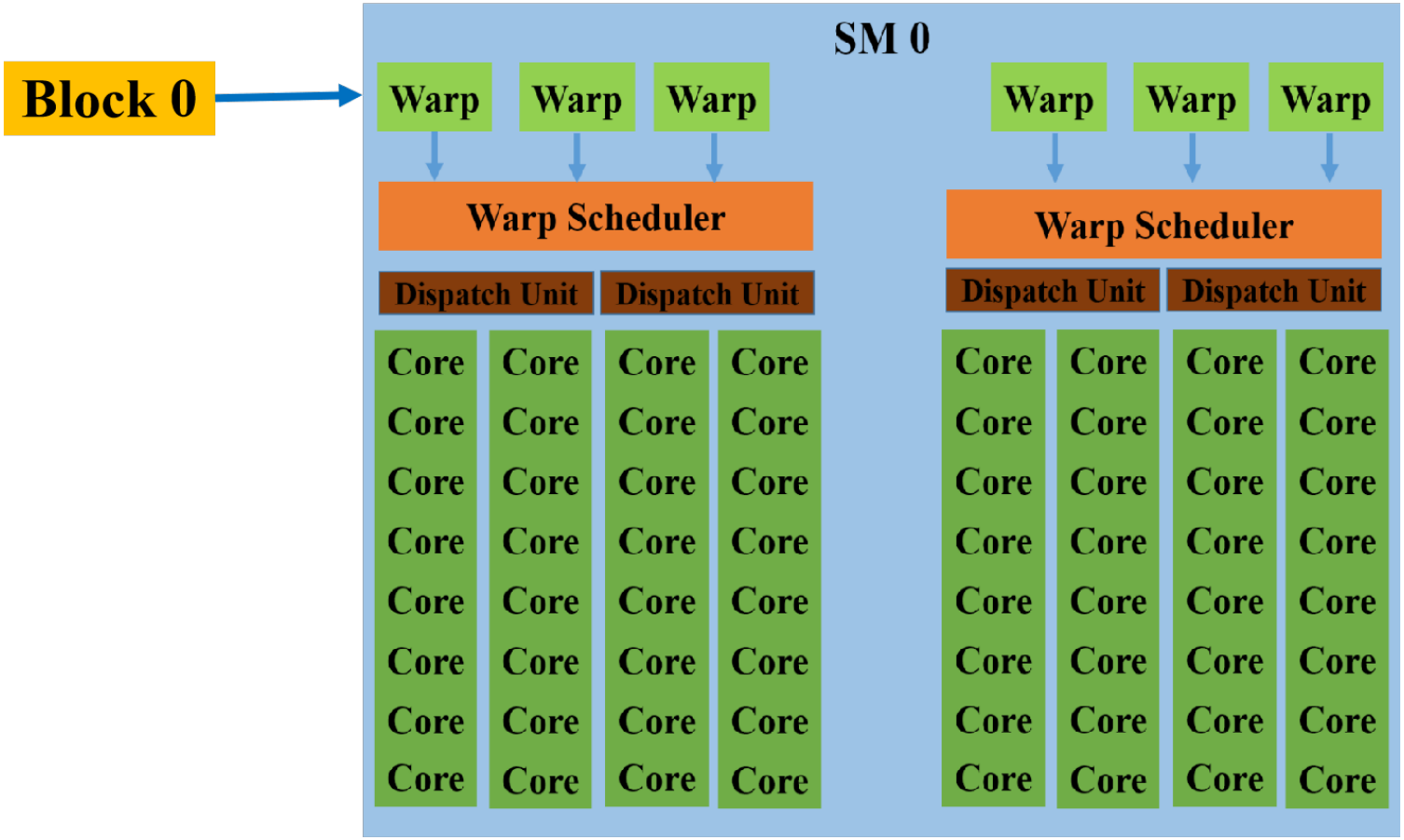
线程与硬件的关系


Kernel Grid
Block 0
Block 1
Block 2
Block 3
Block 4
Block 5
Block 6
Block 7
Device with 2 SMs
SM 0
Block 0
Block 2
Block 4
Block 6
SM 1
Block 1
Block 3
Block 5
Block 7

线程与硬件的关系


Kernel Grid
Block 0
Block 1
Block 2
Block 3
Block 4
Block 5
Block 6
Block 7
Device with 4 SMs
SM 0
Block 0
Block 4
SM 1
Block 1
Block 5
SM 2
Block 2
Block 6
SM 4
Block 3
Block 7

线程与硬件的关系


Block 0
SM 0
Warp
Warp
Warp
Warp
Warp
Warp
Warp Scheduler
Warp Scheduler
Dispatch Unit
Dispatch Unit
Dispatch Unit
Dispatch Unit
Core

线程与硬件的关系

GPU	Kepler GK180	Maxwell GM200	Pascal GP100	Volta GV100
Compute Capability	3.5	5.2	6	7
Threads / Warp	32	32	32	32
Max Warps / SM	64	64	64	64
Max Threads / SM	2048	2048	2048	2048
Max Thread Blocks / SM	16	32	32	32
Max 32-bit Registers / SM	65536	65536	65536	65536
Max Registers / Block	65536	32768	65536	65536
Max Registers / Thread	255	255	255	2551
Max Thread Block Size	1024	1024	1024	1024
FP32 Cores / SM	192	128	64	64
Ratio of SM Registers to FP32 Cores	341	512	1024	1024
Shared Memory Size / SM	16 KB/32 KB/ 48 KB	96 KB	64 KB	Configurable up to 96 KB

https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf

线程与硬件的关系

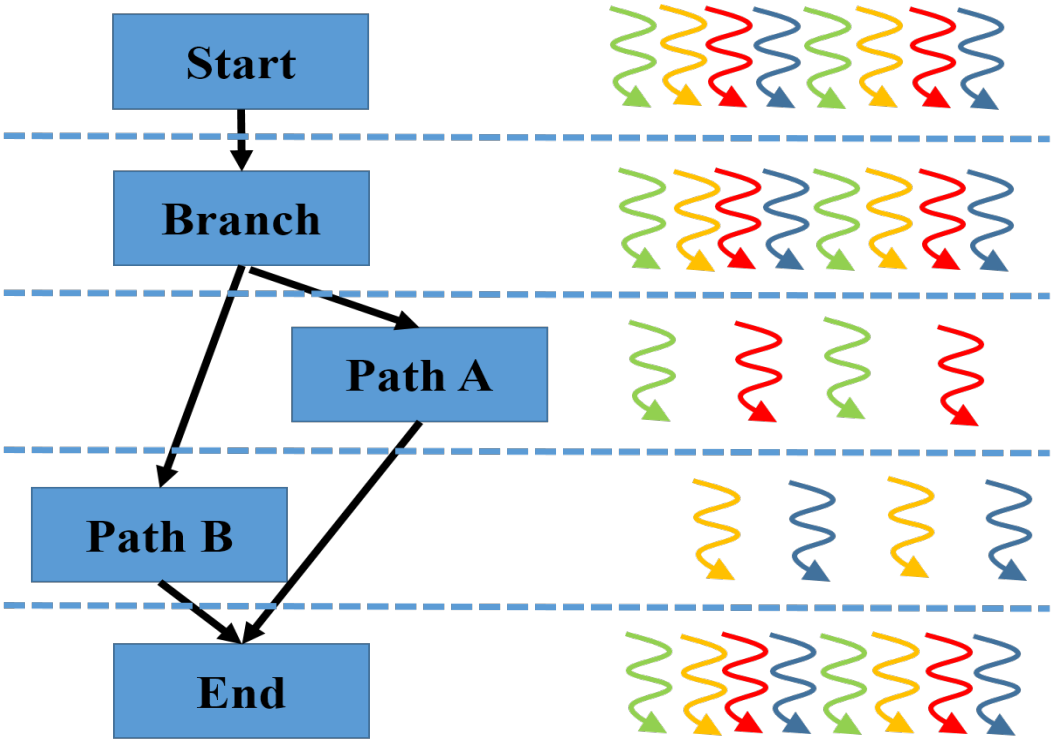
- 在任何给定时间，warp 中的所有线程都必须执行相同的指令，但这会产生分支分歧（branch divergence）
- divergence 示例：

```
if (threadIdx.x % 2==0) { … }
else { … }
```

- 为block中的线程创建两个不同的控制路径

线程与硬件的关系

```
if (threadIdx.x % 2==0) { … }
else { … }
```



线程与硬件的关系

- 如何避免branch divergence?

```
if (threadIdx.x / WARP_SIZE ==0) { … }
else { … }
```

- 还为一个 block 中的线程创建两个不同的控制路径，但粒度是warp size的整数倍，整个 warp 做同样的工作
- 将具有相同分支行为（即执行相同分支路径）的线程放在同一个warp中，这样warp中的线程就可以并行执行，从而达到减少Branch Divergence、提高性能的目的

Vector Addition 版本1

```
add<<< N, 1 >>>();
```

- 尖括号内部参数1代表Block数量，参数2代表每个Block中Thread数量，可以是1D、2D或3D
- 利用内置变量blockIdx.x获取Block一维索引

```
__global__ void add(int *a, int *b, int *c) {
        c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];
}
```

- 通过使用blockIdx.x对数组进行索引，每个Block并行处理不同的索引

Block 0	Block 1	Block 2	Block 3
c[0] = a[0] + b[0];	c[1] = a[1] + b[1];	c[2] = a[2] + b[2];	c[3] = a[3] + b[3];

Vector Addition 版本1

- main()函数

```
#define N 512
int main(void) {
    int *a, *b, *c;          // host copies of a, b, c
    int *d_a, *d_b, *d_c;    // device copies of a, b, c
    int size = N * sizeof(int);

    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Alloc space for host copies of a, b, c and setup input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);
```

Vector Addition 版本1

- main()函数

```
    // Copy inputs to device
    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    // Launch add() kernel on GPU with N blocks
    add<<<N,1>>>(d_a, d_b, d_c);

    // Copy result back to host
    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

    // Cleanup
    free(a); free(b); free(c);
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}
```

Vector Addition 版本2

```
add<<< 1, N >>>();
```

- 也可以将Block并行替换为Thread并行
- 利用内置变量threadIdx.x代替blockIdx.x

```
__global__ void add(int *a, int *b, int *c) {
        c[threadIdx.x] = a[threadIdx.x] + b[threadIdx.x];
}
```

Vector Addition 版本2

- main()函数

```
#define N 512
int main(void) {
    int *a, *b, *c;                    // host copies of a, b, c
    int *d_a, *d_b, *d_c;              // device copies of a, b, c
    int size = N * sizeof(int);

    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Alloc space for host copies of a, b, c and setup input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);
```

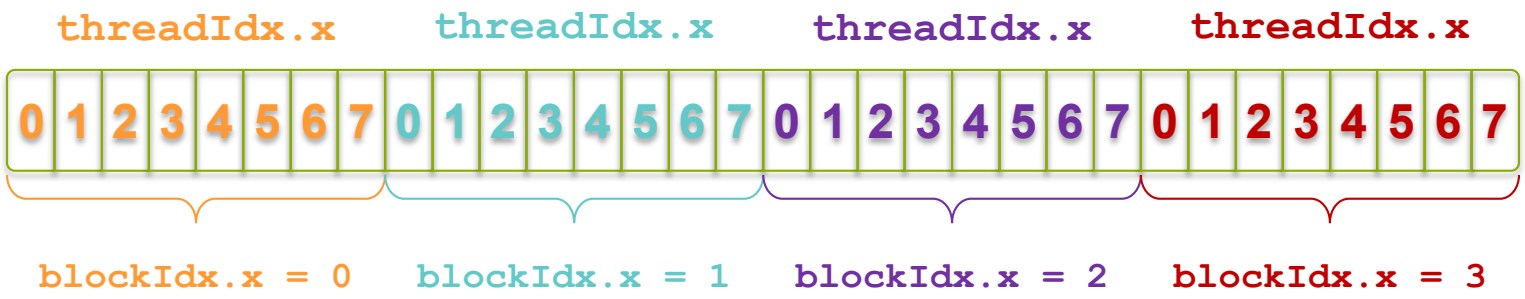
Vector Addition 版本2

- main()函数

```
    // Copy inputs to device
    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    // Launch add() kernel on GPU with N threads
    add<<<1,N>>>(d_a, d_b, d_c);

    // Copy result back to host
    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

    // Cleanup
    free(a); free(b); free(c);
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}
```

Vector Addition 版本3

- 硬件限制，每个Block中线程数有上限（一般为1024)
- 当N的规模变的很大时，就需要联合设置多Block和多Thread
- 不能再简单使用`blockIdx.x`或`threadIdx.x`

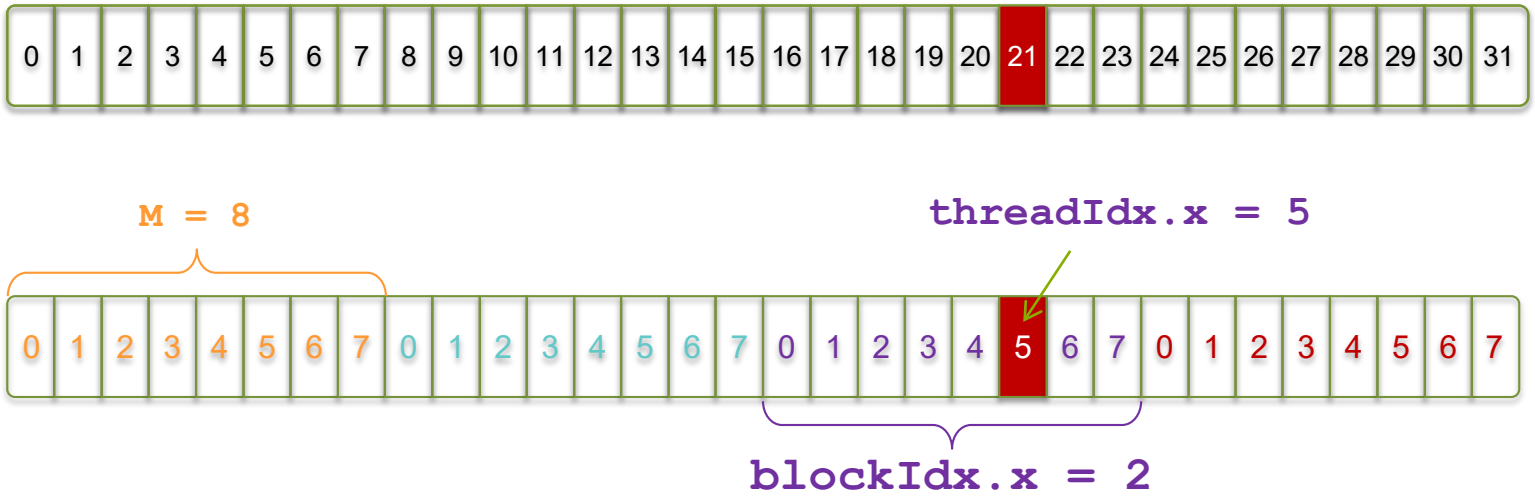


- 对于M个线程/Block，每个线程对应的数组唯一索引由下式给出：

```
int index = threadIdx.x + blockIdx.x * M;
```

Vector Addition 版本3

- **问题：哪个线程对红色标注的数组元素进行操作?**




```
int index = threadIdx.x + blockIdx.x * M;
          =       5     +     2      * 8;
          = 21;
```

Vector Addition 版本3

- 使用内置变量blockDim.x获取每个Block中的线程数

```
int index = threadIdx.x + blockIdx.x * blockDim.x;
```

- add()核函数

```
__global__ void add(int *a, int *b, int *c) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    c[index] = a[index] + b[index];
}
```

Vector Addition 版本3

- main()函数

```
#define N (1024*1024)
#define THREADS_PER_BLOCK 256
int main(void) {
    int *a, *b, *c;                 // host copies of a, b, c
    int *d_a, *d_b, *d_c;           // device copies of a, b, c
    int size = N * sizeof(int);

    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Alloc space for host copies of a, b, c and setup input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);
```

Vector Addition 版本3

- main()函数

```
    // Copy inputs to device
    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    // Launch add() kernel on GPU
    add<<<N/THREADS_PER_BLOCK,THREADS_PER_BLOCK>>>(d_a, d_b, d_c);

    // Copy result back to host
    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

    // Cleanup
    free(a); free(b); free(c);
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}
```

Vector Addition 版本4

- 处理任意向量大小，怎么改进?
- 避免线程的访问超出数组的界限

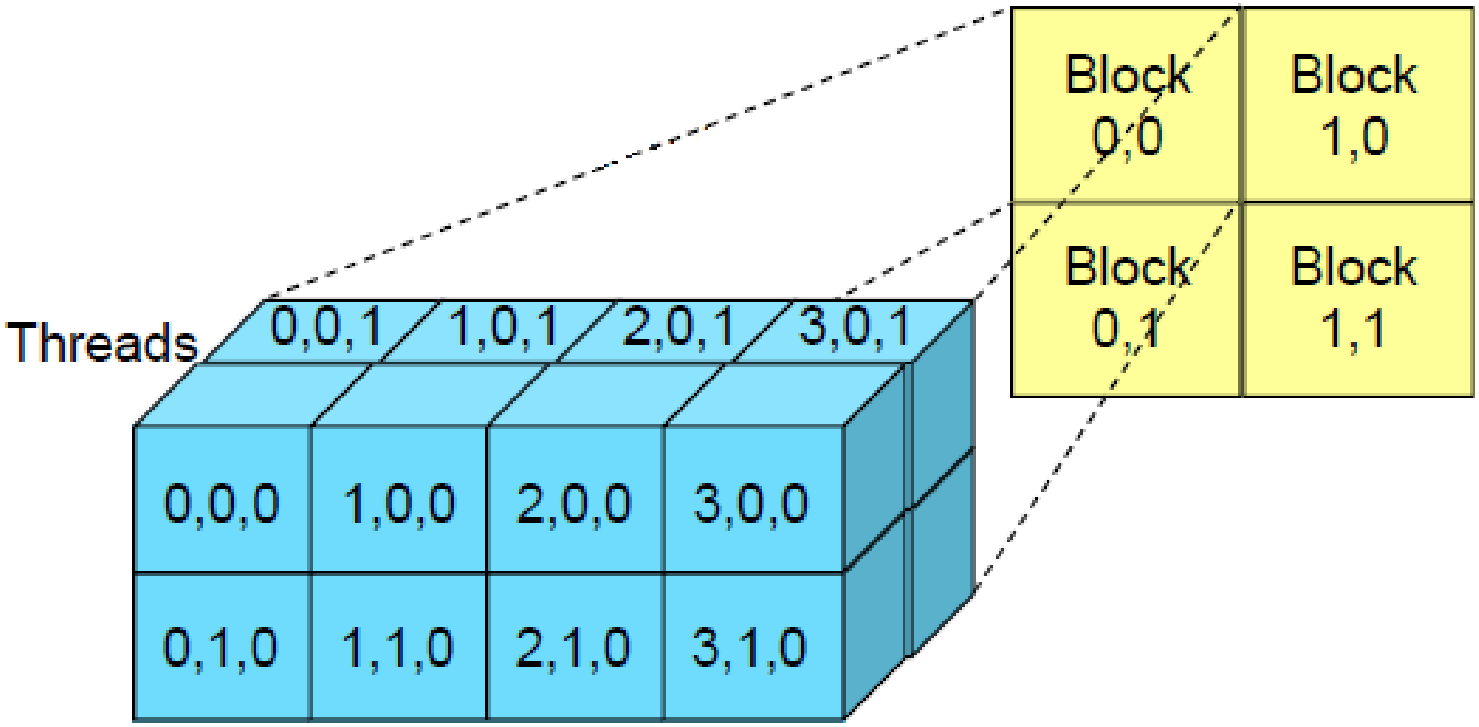
```
__global__ void add(int *a, int *b, int *c, int n) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    if (index < n)
        c[index] = a[index] + b[index];
}
```

- 更新kernel启动语句

```
add<<<(N + M-1) / M,M>>>(d_a, d_b, d_c, N);
```

内置dim3类型

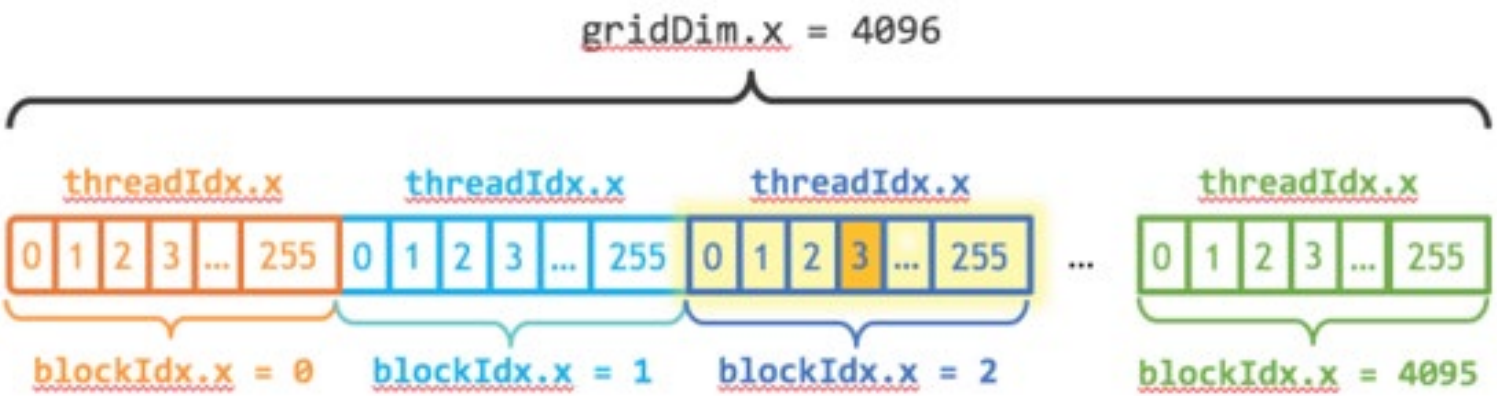
- **定义grid和thread block的组织**
  - `dim3 dimGrid(2, 2);`
  - `dim3 dimBlock(4, 2, 2);`
  - `kernelFunction<<< dimGrid, dimBlock>>>(…);`



内置索引变量

- 利用threadIdx确定线程块中线程的索引，即线程所在block中各个维度上的线程号
  - threadIdx.x threadIdx.y threadIdx.z
- 利用blockIdx确定网格中线程块的索引，即block所在grid中各个维度上的块号
  - blockIdx.x blockIdx.y blockIdx.z
- 利用gridDim确定网格的维度
  - gridDim.x gridDim.y gridDim.z
- 利用blockDim确定线程块的维度
  - blockDim.x blockDim.y blockDim.z

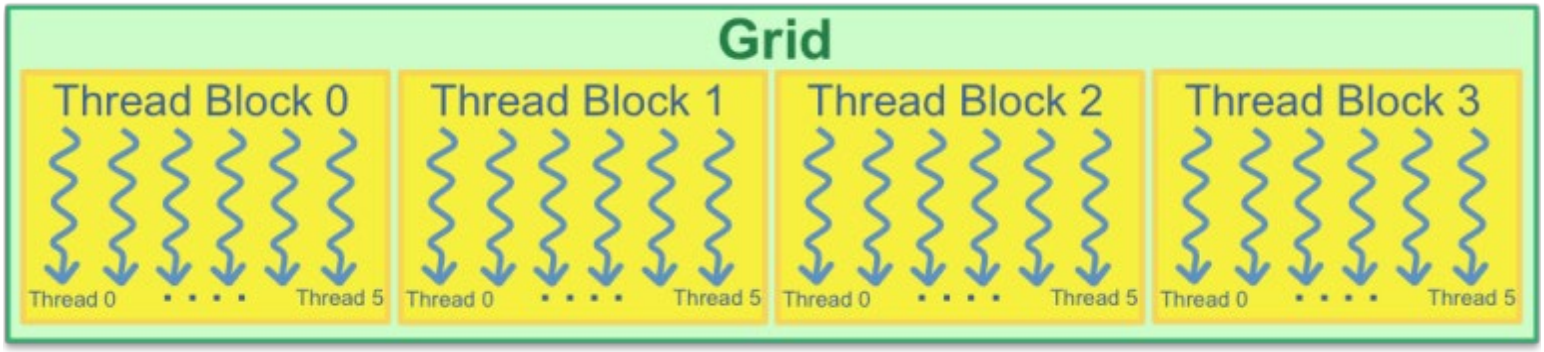
Grids and Thread Blocks




```
index = blockIdx.x  * blockDim.x + threadIdx.x

index =    (2)      *   (256)    +     (3)      = 515
```

Grids and Thread Blocks

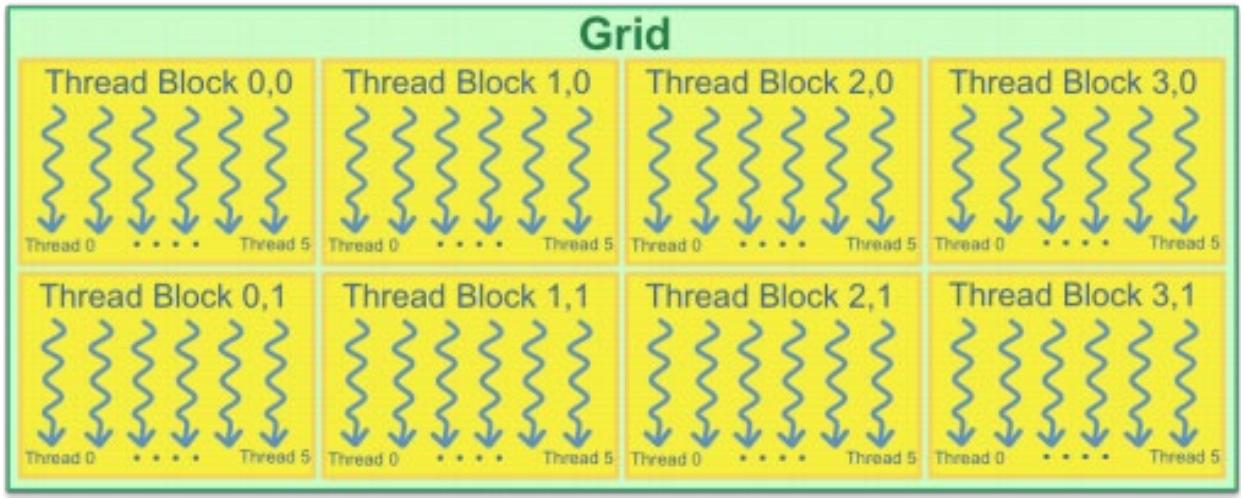
- **A 1D Grid of 1D Blocks**




```
int threadid = blockIdx.x * blockDim.x + threadIdx.x;
```

Grids and Thread Blocks

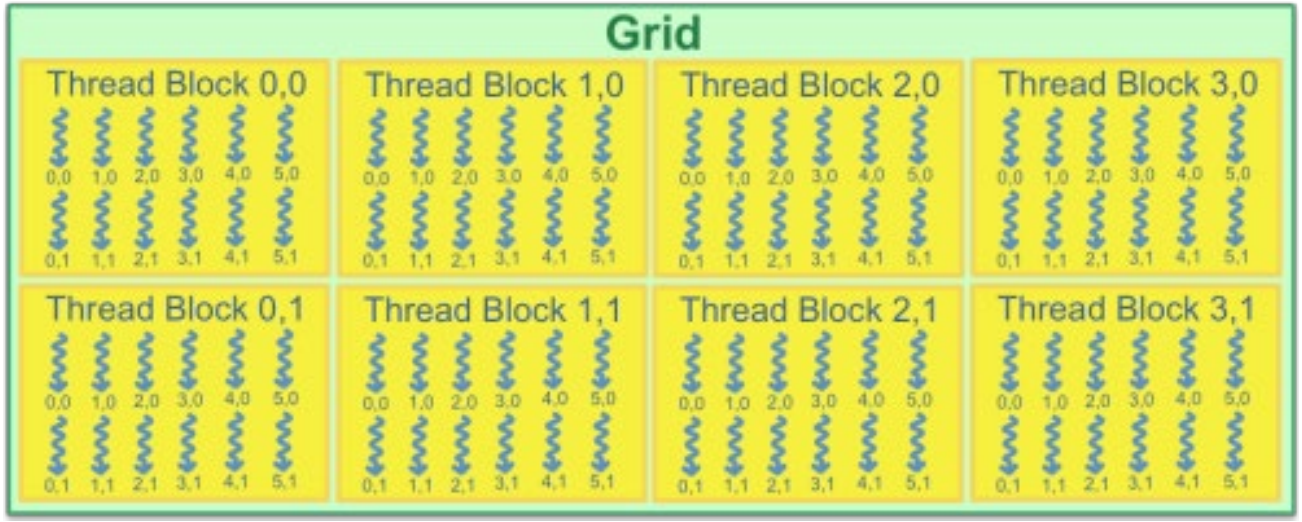
- **A 2D Grid of 1D Blocks**




```
int blockId = blockIdx.y * gridDim.x + blockIdx.x;
int threadId = blockId * blockDim.x + threadIdx.x;
```

Grids and Thread Blocks

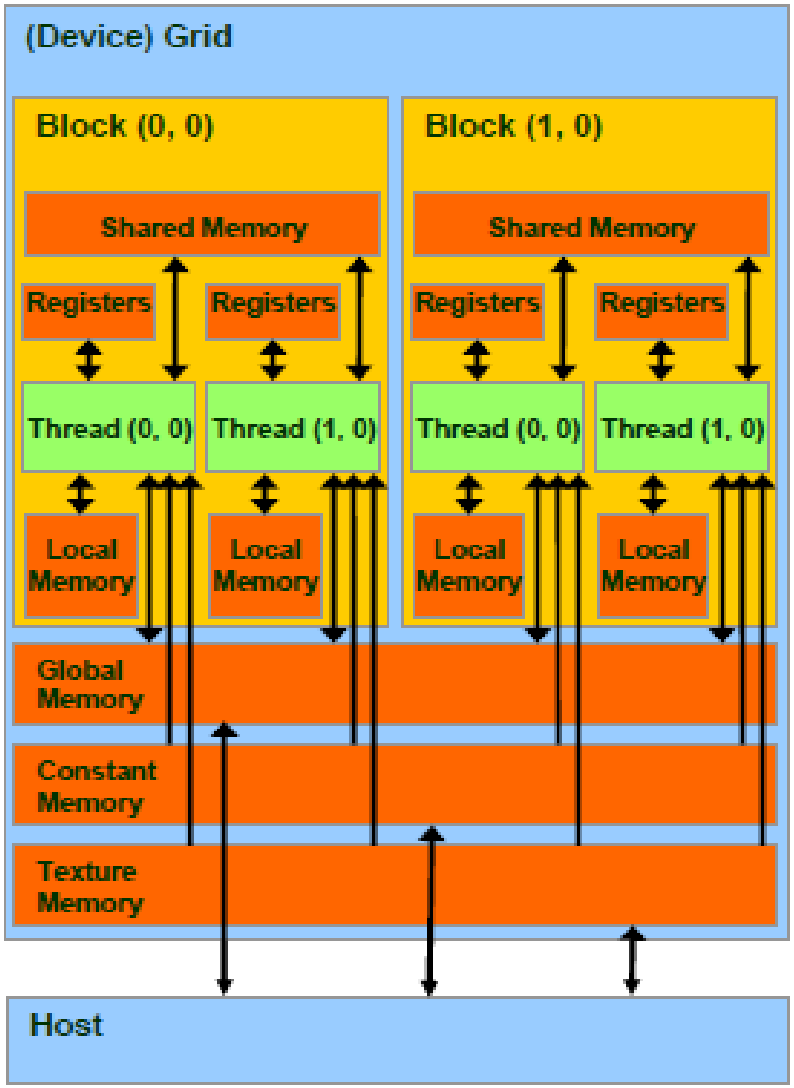
- **A 2D Grid of 2D Blocks**



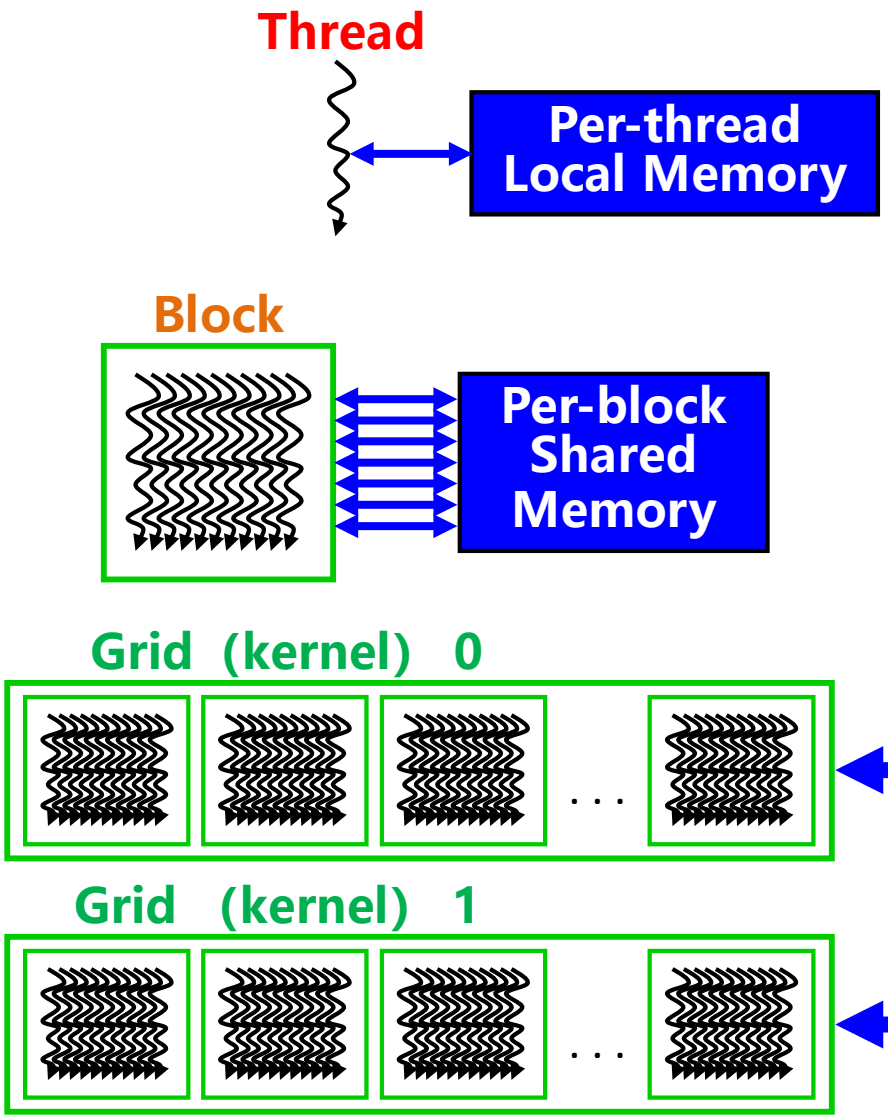

```
int blockId = blockIdx.x + blockIdx.y * gridDim.x;
int threadId = blockId * (blockDim.x * blockDim.y)
        + (threadIdx.y * blockDim.x) + threadIdx.x;
```

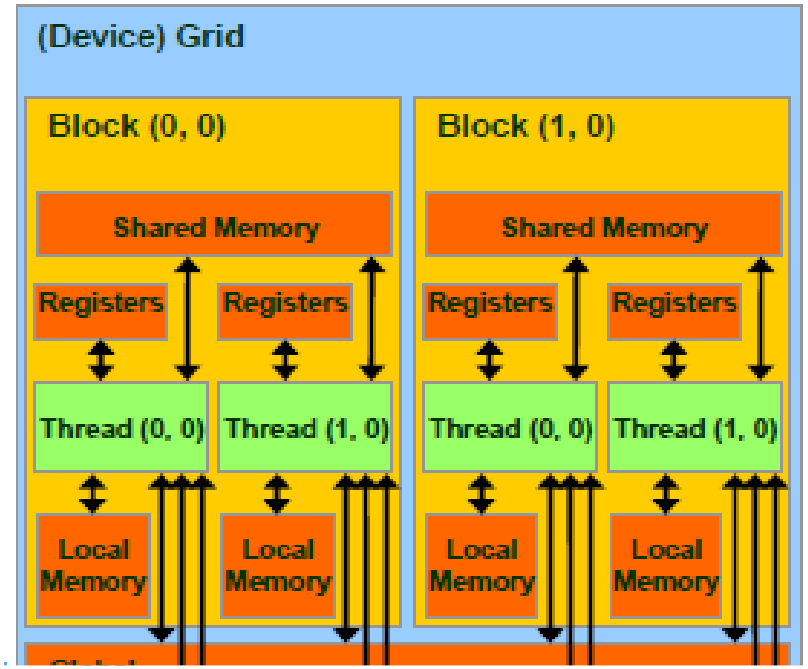
GPU内存的类型

- R/W per-thread registers
  - 1-cycle latency
- R/W per-thread local memory
  - Slow – register spilling to global memory
- R/W per-block shared memory
  - 1-cycle latency
  - But bank conflicts may drag down
- R/W per-grid global memory
  - ~500-cycle latency
  - But coalescing accessing could hide latency
- Read only per-grid constant and texture memories
  - ~500-cycle latency
  - But cached



线程与内存的关系


Thread
Per-thread Local Memory
Block
Per-block Shared Memory
Grid (kernel) 0
. . .
Grid (kernel) 1
. . .
Per-device Global Memory
Sequential Kernels


(Device) Grid
Block (0, 0)
Block (1, 0)
Shared Memory
Shared Memory
Registers
Registers
Registers
Registers
Thread (0, 0)
Thread (1, 0)
Thread (0, 0)
Thread (1, 0)
Local Memory
Local Memory
Local Memory
Local Memory

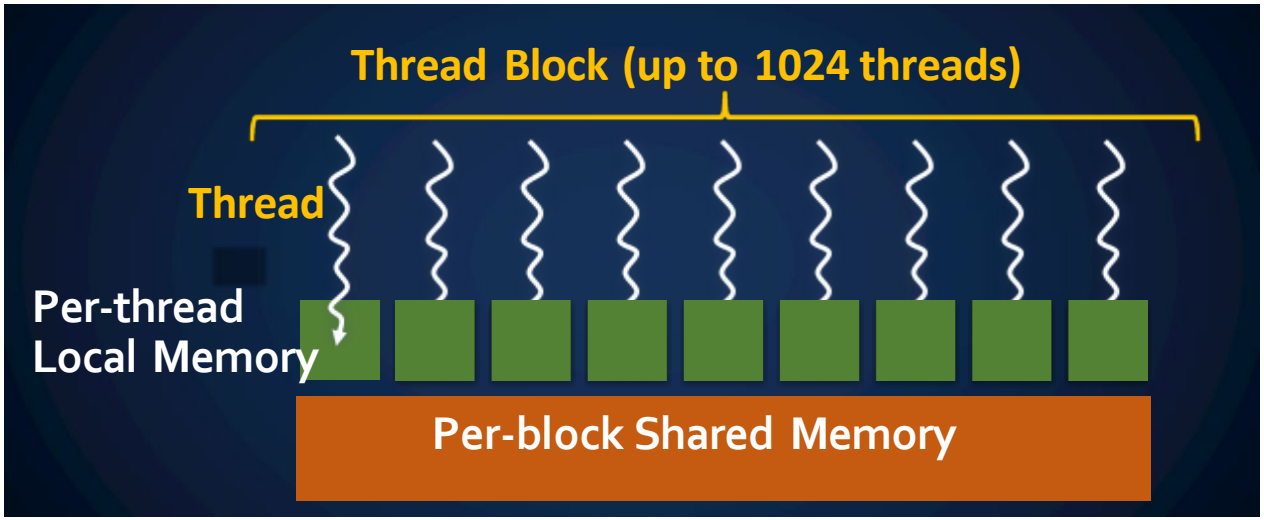
多层次并行策略

- 为什么同时使用线程块（blocks）和线程（threads），这种多层次并行策略?

```
kernel_call<<<blocks,threads>>>(...);
```

- 每个block的线程最大数量有限制
  - 使用单个块里面的线程无法适用于大规模的数组处理
- 同一线程块内的线程可以共享高速的shared memory
  - 线程块之间的数据共享很慢

Shared Memory

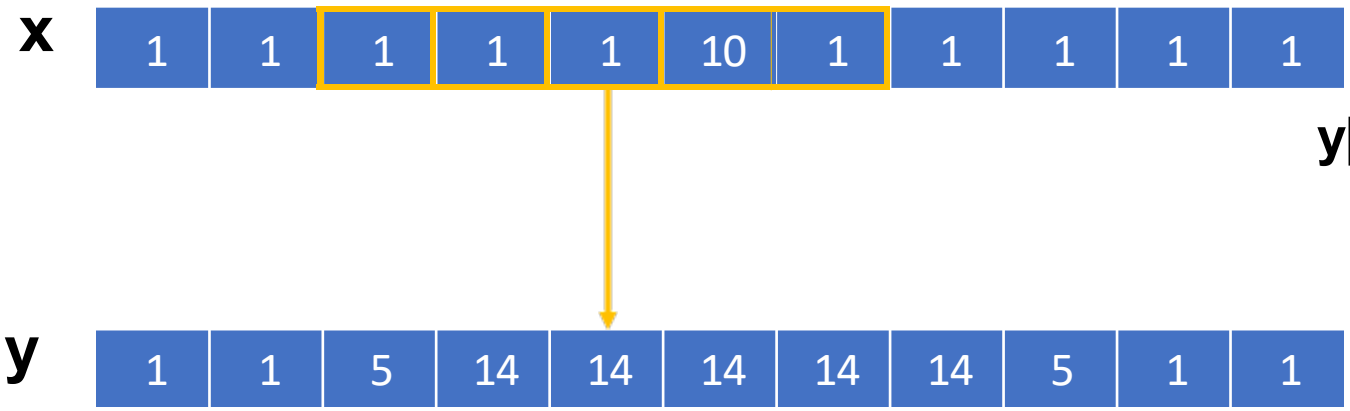


- 使用__shared__进行声明，按线程块分配
- 是快速片上存储器，用户管理
- 对其他线程块中的线程不可见，同一线程块内的线程共享

变量内存空间说明符

- __shared__，变量储存于GPU上thread block内的Shared Memory空间，只能被thread block内的线程存取
- __device__，变量储存于GPU上的Global Memory空间，可被grid中所有线程存取
- __constant__，变量储存于GPU上的Constant Memory空间，可被grid中所有线程存取
- 无修饰（Local变量），储存于SM内的寄存器和Local Memory，Thread私有

1D Stencil



y[i] = x[i] +
x[i-2] + x[i-1] +
x[i+2] + x[i+1]

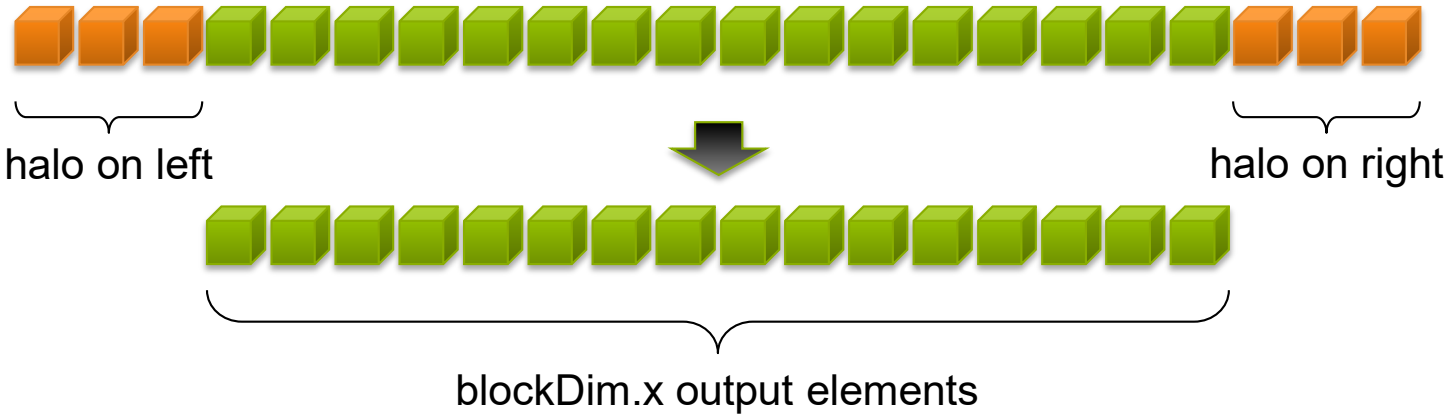
1D 5-point stencil (with a “radius” of 2)

1D Stencil

- 每个线程处理一个输出元素
  - 每个block处理`blockDim.x`个元素
- 输入元素被读取多次
- 输入数据的重复使用情况：
  - radius为2，每个输入元素读取5次
  - radius为3，每个输入元素读取7次

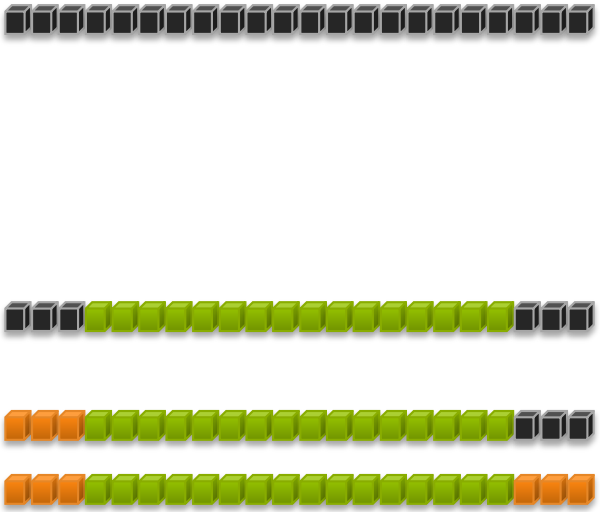
1D Stencil

- 策略：利用shared memory缓存数据
- 从global memory 读取（`blockDim.x+2*radius`）个输入元素到shared memory
- 计算`blockDim.x`输出元素
- 将`blockDim.x`输出元素写入global memory



1D Stencil Kernel

```
__global__ void stencil_1d(int *in, int *out) {
  __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];
  int gindex = threadIdx.x + blockIdx.x * blockDim.x;
  int lindex = threadIdx.x + RADIUS;

  // Read input elements into shared memory
  temp[lindex] = in[gindex];
  if (threadIdx.x < RADIUS) {
    temp[lindex - RADIUS] = in[gindex - RADIUS];
    temp[lindex + BLOCK_SIZE] =
      in[gindex + BLOCK_SIZE];
  }
```



1D Stencil Kernel

```
  // Apply the stencil
  int result = 0;
  for (int offset = -RADIUS ; offset <= RADIUS ; offset++)
    result += temp[lindex + offset];

  // Store the result
  out[gindex] = result;
}
```

1D Stencil

- 上述Stencil Kernel有什么问题?
- 假设线程15在线程0获取halo之前读取halo…
- 数据竞争（Data Race）！

```
temp[lindex] = in[gindex];                Store at temp[18]
if (threadIdx.x < RADIUS) {
  temp[lindex - RADIUS] = in[gindex - RADIUS];    Skipped, threadIdx > RADIUS
  temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
}

int result = 0;
result += temp[lindex + 1];               Load from temp[19]
```

线程同步

```
void __syncthreads();
```

- 同步一个线程块（Block）中的所有线程
- 用于防止RAW/WAR/WAW问题
- 保证所有线程必须到达barrier
  - 在条件代码中，条件在整个块中必须是一致的

```
if (threadIdx.x < 10)
{
    __syncthreads();
}
```

线程同步

```
void __syncthreads();
```

- 同步一个线程块（Block）中的所有线程
- 用于防止RAW/WAR/WAW问题
- 保证所有线程必须到达barrier
  - 在条件代码中，条件在整个块中必须是一致的
- kernel与kernel之间有隐式barrier

```
vec_minus<<<nblocks, blksize>>>(a, b, c);
vec_dot<<<nblocks, blksize>>>(c, c);
```

1D Stencil Kernel

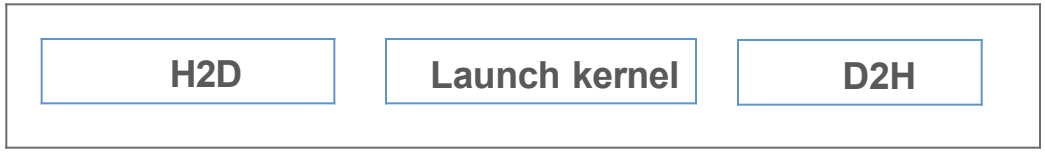
```
__global__ void stencil_1d(int *in, int *out) {
    __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];
    int gindex = threadIdx.x + blockIdx.x * blockDim.x;
    int lindex = threadIdx.x + radius;

    // Read input elements into shared memory
    temp[lindex] = in[gindex];
    if (threadIdx.x < RADIUS) {
        temp[lindex - RADIUS] = in[gindex - RADIUS];
        temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
    }

    // Synchronize (ensure all the data is available)
    __syncthreads();
```

1D Stencil Kernel

```
    // Apply the stencil
    int result = 0;
    for (int offset = -RADIUS ; offset <= RADIUS ; offset++)
        result += temp[lindex + offset];

    // Store the result
    out[gindex] = result;
}
```

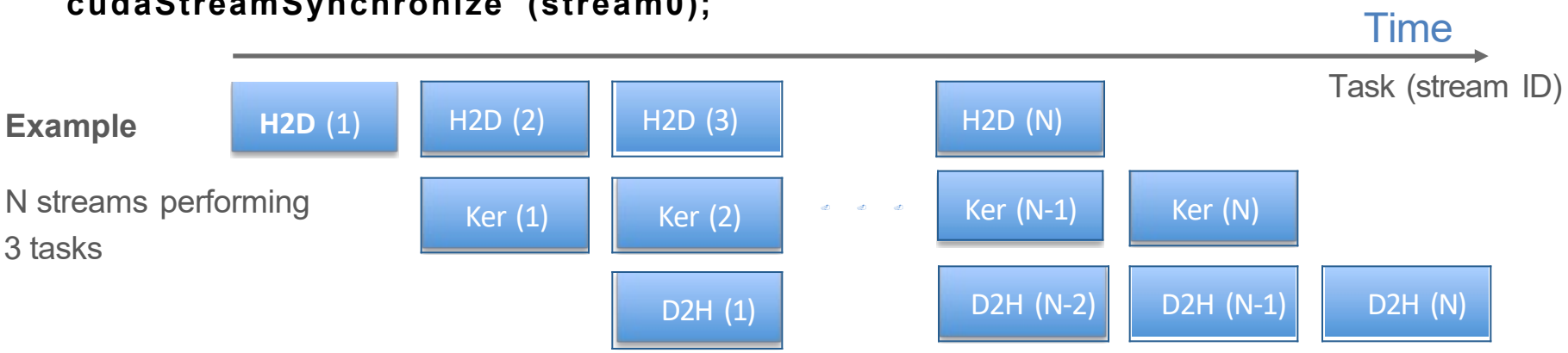
CUDA Stream

- **Stream，定义为按顺序执行的设备操作序列，包括kernel执行和数据传输**



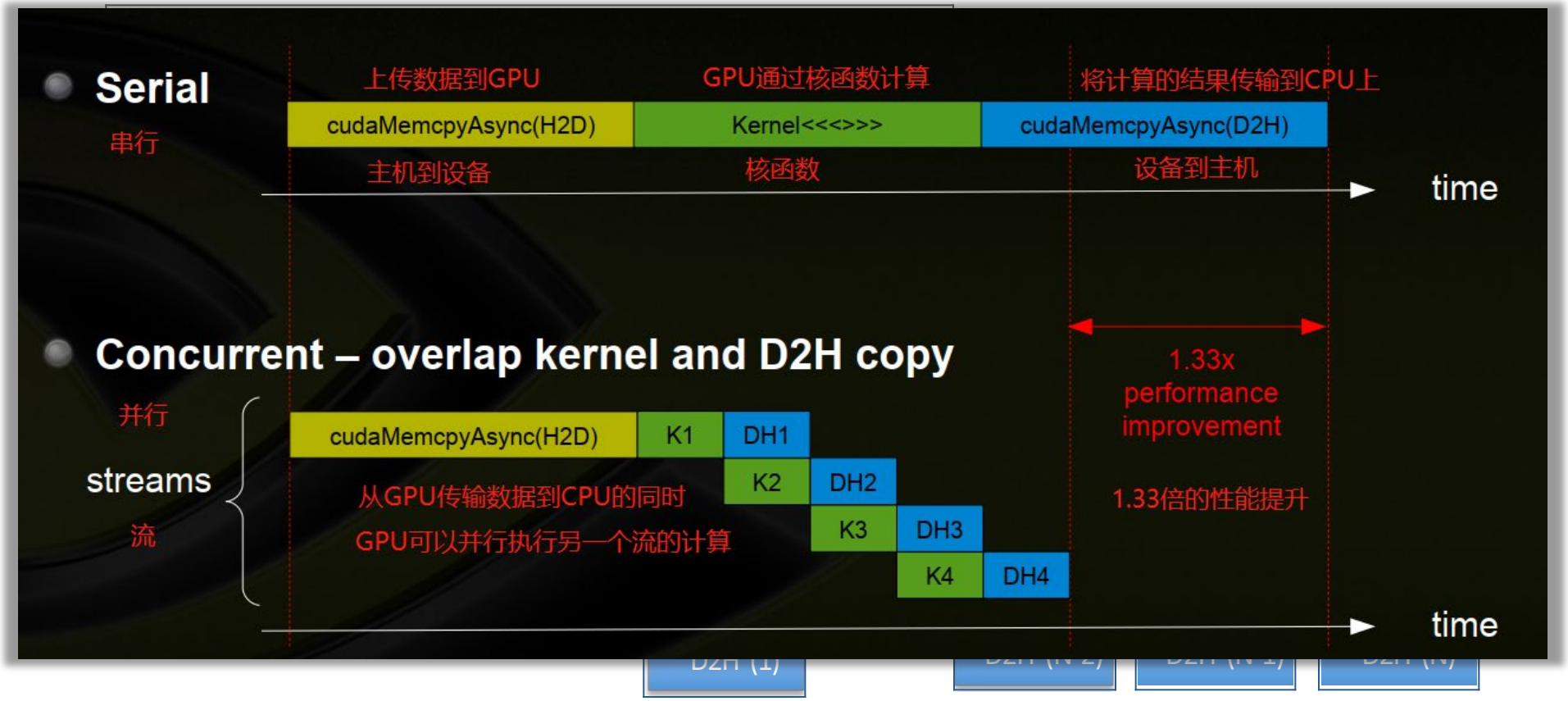
Stream 1

```
cudaStream_t stream0, stream1;

cudaStreamCreate (&stream0);

cudaMemCopyAsync (..., stream0);   someKernel<<<..., stream0>>>();
cudaMemCopyAsync (..., stream1);   someKernel<<<..., stream1>>>();
cudaStreamSynchronize (stream0);
```



CUDA Stream

- **Stream，定义为按顺序执行的设备操作序列，包括kernel执行和数据传输**



获取GPU信息

```
#include <stdio.h>
int main() {
   int noOfDevices;
   /*get no. of device*/
   cudaGetDeviceCount(&noOfDevices);
   cudaDeviceProp prop;
   for(int i = 0;i<noOfDevices;i++) {
      /*get device properties */
      cudaGetDeviceProperties(&prop,i);
      printf("Device Name:\t%s\n",prop.name);
      printf("Total global memory:\t%ld\n ",
                          prop.totalGlobalMem);
      printf (”No . of SMs:\t %d\n " ,
                          prop.multiProcessorCount);
      printf ("Shared memory / SM:\t %ld\n " ,
                          prop.sharedMemPerBlock);
      printf("Registers / SM:\t %d\n " ,
                          prop.regsPerBlock);
  }
  return 0;
}
```

CUDA函数:
cudaGetDeviceCount
cudaGetDeviceProperties

编译

```
nvcc whatDevice.cu –o whatDevice
```

输出

```
Device Name:            Tesla C2050
Total global memory:    2817720320
No. of SMs:             14
Shared memory / SM:     49152
Registers / SM:         32768
```

For more properties see
struct cudaDeviceProp

For details see CUDA Reference Manual

CUDA计时

```
int  main()  {
        cudaEvent_t start, stop;
        float time;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);
        cudaEventRecord(start,0);


someKernel<<<grids,blocks,0,0>>>(...);

        cudaEventRecord(stop,0);
        cudaEventSynchronize(stop);


cudaEventElapsedTime(&time,start,stop);
        cudaEventDestroy(start);
        cudaEventDestroy(stop);
        printf("Elapsed time %f
sec\n", time*.001);
```

CUDA Event API Timer are,

- **OS independent**
- **High resolution**
- **Useful for timing asynchronous calls**

Ensures kernel execution has completed

Standard CPU timers will not measure the timing information of the device.

其他

- Constant Memory
- Texture Memory
- Graphics Interoperability
- Atomics
- Streams
- CUDA C On Multiple GPUs

官方指南：https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#programming-interface

Reference

- CUDA C++ Programming Guide, https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#programming-interface
- CUDA Toolkit Documentation, https://docs.nvidia.com/cuda/index.html
- Blaise Barney, Lawrence Livermore National Laboratory, Introduction to Parallel Computing, https://computing.llnl.gov/tutorials/parallel_comp/